



Lecture 1: What is Svelte?

INTERVIEW QUESTION

[> CORE](#)

What is Svelte, and how does it differ from frameworks like React and Vue?

You've heard Svelte "has no virtual DOM" — but what does that actually mean, and why does it matter for performance?

When building web applications, developers want to write declarative code (saying *what* the UI should look like based on state) instead of imperative code (writing out the exact steps to update the DOM). Frameworks like React and Vue achieve this by doing heavy work in the browser while the user is using the app. They load a large "runtime" engine, keep a memory representation of the entire UI (the Virtual DOM), and constantly compare the old state to the new state to figure out what changed.

Svelte takes a fundamentally different approach. It is not just a library you include in your webpage; it is a **compiler**.

The Compiler Advantage

Instead of doing the work in the browser, Svelte does the heavy lifting on your computer during the build step. It analyzes your `.svelte` files (which contain HTML, CSS, and JavaScript) and compiles them into highly optimized, standard JavaScript before the code is ever sent to the user.

KEY TAKEAWAY

What the DOM is to HTML, the AST is to code.

Just as the browser parses a flat HTML string into a Document Object Model (DOM) tree so it can manipulate the web page, a compiler parses a flat string of code into an Abstract Syntax Tree (AST) so it can analyze and manipulate the program. Svelte reads your declarative components, builds an AST, and figures out exactly how the state connects to the DOM.

Because Svelte knows exactly what can change at compile time, it generates vanilla JavaScript that directly targets those specific DOM nodes.



App.svelte

```
1  <script>
2    let count = $state(0);
3
4    function increment() {
5      count += 1;
6    }
7  </script>
8
9  <button onclick={increment}>
10    Clicks: {count}
11  </button>
```

*Directly updates the **TEXT** node, no diffing required*

No Virtual DOM

INTERVIEW TIP

To impress the interviewer: During an interview, most candidates will simply state that "Svelte doesn't use a Virtual DOM." If you want to show deep technical background, explain *why* it doesn't need one — this proves you understand the mechanics, not just the marketing.

React and Vue need a Virtual DOM because their runtime doesn't know what changed until it diffs the old virtual tree against the new one. Svelte doesn't need a Virtual DOM because its compiler has already wired the variable `count` directly to the specific text node inside the `<button>`. When `count` changes, Svelte's generated code surgically updates that exact spot.

By compiling away the framework, Svelte delivers smaller bundle sizes, faster initial load times, and eliminates the runtime overhead of diffing trees in memory.

Summary

Svelte shifts the heavy lifting of building user interfaces from the browser (runtime) to your local machine (compile time). It takes declarative components and compiles them into highly optimized JavaScript that updates the DOM surgically, without needing an intermediate Virtual DOM.

	REACT / VUE (Runtime Library)	SVELTE (Build-Time Compiler)
Architecture	Ships a framework engine to the browser	Compiles components to vanilla JS at build time
State Updates	Virtual DOM diffing	Direct, surgical DOM updates
Bundle Size	Includes the framework runtime engine	Contains only the generated vanilla JavaScript

	REACT / VUE (Runtime Library)	SVELTE (Build-Time Compiler)
Performance	Overhead from keeping a virtual tree in memory	Bare-metal performance by updating nodes directly

Lecture 2: Setting up a Svelte Project

INTERVIEW QUESTION

[> CORE](#)

How do you create and run a new Svelte project?

You want to start a Svelte app today. What command do you run, what does it give you, and how do you see it in the browser?

Starting a new project in modern Svelte is done entirely through a dedicated command-line tool. Unlike the early days where you might have manually wired up a bundler, the Svelte team now provides an official, zero-configuration pathway.

The Modern Standard: `sv create`

The easiest and official way to create a Svelte project is to use the `sv` CLI. This tool generates a complete, ready-to-run skeleton for your application.



When starting a new project, you need a place to put all your files, your settings, and your codebase. Should we manually create every single folder? Should we create an `index.html`, then a `styles.css`, and manually link them together one by one? This is a decision about how we build the foundation of our app. We need a way to instantly generate the correct folders and baseline files so we don't make mistakes setting up the architecture. This automated generation of the foundation is called **scaffolding** an application. When you run `npx sv create`, you are scaffolding a **SvelteKit** application.

KEY TAKEAWAY

SvelteKit is not an optional add-on; it is the official operating system for Svelte.

SvelteKit is the official framework built *on top of* Svelte (powered by Vite under the hood). To understand why it exists, we must look at what a full app needs.

When you create a full app, you need to make some decisions on how to present new pages. Should we name new pages automatically with "p", for example, `/p/1` is page 1, and `/p/2` is page 2? Or maybe we should write `/page-1` for page 1? This is a decision about the architecture of our app. We have to make sure that when a specific path is visited by the user in the browser (e.g., `mywebsite.com/page/1`), a specific part of our app is activated. This matching mechanism is called the **router** of the app. The router makes sure to match all the paths with the proper components.

SvelteKit provides this router for you. Just as an operating system provides the file system and networking for software, SvelteKit provides the architecture for your Svelte components to live in.

Furthermore, when a user clicks a link to go to a new page, the browser usually talks to a server, downloads a brand new HTML document, and the screen flashes white while loading. Do we always want that flash? What if we just download the entire website's structure once, and then use JavaScript to swap out the content instantly when the user clicks a link? This is a decision about how our app behaves. We need a way for the app to feel as fast as a desktop program without constantly asking the server for new pages. An app that loads once and swaps content dynamically without refreshing is called a **Single-Page Application (SPA)**.

Even if you only want to build a simple SPA and don't know the rest of SvelteKit yet, the team recommends using the `npx sv create` command. You can safely ignore the advanced features until you need them.

INTERVIEW TIP

To impress the interviewer: Mention that `npx sv create` scaffolds a SvelteKit project by default. Point out that the ecosystem has shifted away from standalone Svelte templates, and that SvelteKit is now the recommended baseline even for simple client-side apps.

The Development Server

After running `npm run dev`, Vite spins up a local web server (usually at `http://localhost:5173`). This server watches your files for changes. When you edit a component, Vite leverages Hot Module Replacement (HMR) to instantly push the updated compiled code to your browser without requiring a full page refresh.

The Standalone Alternative

If you absolutely must avoid SvelteKit (perhaps you are integrating Svelte into a legacy backend that serves its own HTML), you can use Vite directly:



A terminal window with a title bar containing three window control buttons and the label "Terminal". The terminal shows a single command: `1 npm create vite@latest myapp -- --template svelte`. Below the command, a callout box contains the text *Creates a*, a **STANDALONE** badge, and *Svelte setup*.

```
1 npm create vite@latest myapp -- --template svelte
```

Creates a **STANDALONE** *Svelte setup*

This bypasses SvelteKit entirely, giving you a bare-bones Svelte project. However, you will have to handle routing and server-side rendering yourself.

Summary

To start a new Svelte project, run `npx sv create myapp` . This command scaffolds a complete application structure powered by SvelteKit and Vite. After installing dependencies with `npm install` , you use `npm run dev` to start a local development server that instantly reflects your code changes in the browser.

	SVELTE CLI (<code>sv create</code>) (SvelteKit Powered)	VITE CLI (<code>create vite</code>) (Standalone SPA)
Output	Full-stack SvelteKit app skeleton	Client-side only Svelte skeleton
Routing	Built-in file- based routing	None (requires external library)
Recommendation	Official team recommendation for all apps	Useful for legacy integrations or strict SPAs

Lecture 3: The Anatomy of a Svelte Component

INTERVIEW QUESTION

[> CORE](#)

What are the three parts of a `.svelte` file, and what is each one for?

You open a `.svelte` file for the first time and see script, markup, and style all in one place. How do these three pieces fit together?

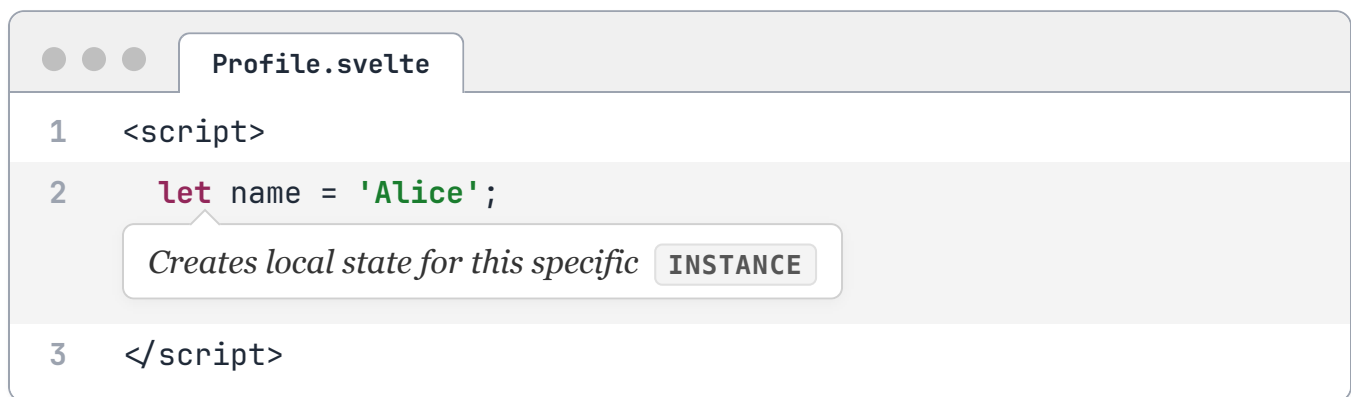
A Svelte component is a self-contained building block for your user interface. It is written in a `.svelte` file that uses a superset of HTML. Inside this single file, you can define three distinct sections. All three sections are entirely optional—you only include what you need.

1. The `<script>` Block (Logic)

The `<script>` block contains the JavaScript (or TypeScript) that controls how your component behaves.

When we define a user profile card in our code, we usually want to show more than one user on the screen at once. Should we copy and paste the profile card code three times? Or do we write it once as a blueprint, and then tell the framework "build me three of these"? This is a decision about how we reuse code. We need a way to stamp out multiple independent copies of the same UI blueprint, where each copy holds its own independent state (like its own name or counter variable). Each independent copy built from the blueprint is called a **component instance**.

The code inside the `<script>` block runs every time a new component instance is created. Any variable you declare here becomes the private, local state for that specific copy.



The `<script module>` Alternative

We know each copy of a component keeps its own private data. But what if all the copies need to talk to each other?

Imagine you are building a webpage with five audio players on it (like a podcast list). When a user clicks "Play" on Player #3, Player #1 needs to pause. You don't want two podcasts playing over each other. But if we build an `AudioPlayer.svelte` component and stamp out 5 copies, we have 5 independent "islands." Player #1 has no idea that Player #3 even exists.

So how do we make the 5 islands talk to each other? Do we pass messages all the way up to the main webpage, and then back down to all the other players telling them to stop? That's exhausting. Do we create a completely separate JavaScript file just to hold one variable called `currentlyPlaying` ? That feels like overkill.

This is a decision about data architecture. We need a way for these 5 component instances to share a single, *communal brain*. We want exactly one variable that all 5 players can look at and update. We need a place inside our `.svelte` file to run code exactly *once*, when the file is first loaded, rather than every time a new player is stamped out.

This shared, run-once area is called the **module script** (written as `<script module>`).

● ● ● **AudioPlayer.svelte**

```
1 <script module>
2   let currentlyPlaying = null;
3 </script>
4
5 <script>
6   let audioSrc = 'podcast.mp3';
7
8   function play() {
9     currentlyPlaying = audioSrc;
10  }
11 </script>
```

The communal brain: shared across ALL 5 players

The local island: runs for every NEW player created

Player 3 updates the shared brain!

Because the module script only runs once, any variable you put inside it is shared across every single copy of that component on the screen.

2. The Markup (Structure)

The markup is the HTML structure of your component. It sits outside of any tag, directly in the body of the `.svelte` file. You can freely reference any variable declared in your `<script>` block inside your markup.

KEY TAKEAWAY

A Svelte file is just HTML with superpowers, not JavaScript with HTML crammed inside it.

If you look at a React component, you are fundamentally writing a JavaScript function that happens to return HTML (via JSX). In Svelte, you are fundamentally writing an HTML file that happens to include logic. This inversion means that any valid HTML file is already a valid Svelte component.

3. The `<style>` Block (Aesthetics)

The `<style>` block contains the CSS for your component.

When we write CSS for our components, we need a way to style elements like a `<p>` tag. Should we just put `p { color: red; }` in our CSS? But what if another component also has a `<p>` tag? Will they all turn red, leaking styles everywhere? Do we need to invent complex class names like `class="my-unique-component-p-tag"` for every single element?

This is a decision about the architecture of our CSS. We need a way to guarantee that the styles written inside a file only affect the elements in that exact file, and nowhere else in the app. This guaranteed isolation is called **scoped styling**. Scoped styling ensures that your CSS cannot accidentally leak out and break other parts of your application.



```
1 <style>
2   p {
3     color: burlywood; /* ✓ safely **SCOPED** to this component only */
4   }
5 </style>
```

INTERVIEW TIP

To impress the interviewer: When asked about `.svelte` files, emphasize that scoping is handled automatically by the compiler. You don't need external tooling like CSS Modules or BEM naming conventions; Svelte simply hashes the classes for you.

Summary

A `.svelte` file consists of three optional parts. The `<script>` block holds the JavaScript logic that runs when a component instance is created, managing its local state. The HTML markup defines the structure of the component and can reference variables from the script. Finally, the `<style>` block contains CSS that is automatically scoped to affect only the markup within that specific file, preventing global style leaks.

	SVELTE COMPONENT (Single-File Anatomy)	REACT COMPONENT (JSX Anatomy)
Foundation	Superset of HTML	JavaScript function
Styling	Built-in, scoped <code><style></code> tags	Requires external CSS-in-JS or CSS Modules
Logic	<code><script></code> block	Function body

Lecture 4: Creating Reactive Variables

INTERVIEW QUESTION

[> CORE](#)

How do you create reactive variables in Svelte?

You change a variable in your script, but the screen doesn't update. Why, and how do you make it update?

When building an application, you often have data that changes over time, like a user's score, a toggled dark mode setting, or text typed into an input field.

The Problem with Plain Variables

Imagine you have a button that increases a counter. You write `let count = 0;` in your script, and in your click function you do `count = count + 1`.

In regular JavaScript, if you change `count = 1`, does the `<h1>` tag displaying that number on the screen automatically update? No. You would have to manually find the HTML element (e.g., `document.getElementById`) and forcefully update its text every single time the number changes. This is exhausting and error-prone.

We need a system where the HTML "listens" to our variables. When the variable changes in the script, the HTML should update itself automatically without us writing manual DOM manipulation code. This automatic synchronization between your variables and your screen is called **reactivity**. Reactivity ensures that your user interface always perfectly reflects the current state of your data.

How to Tell Svelte About Reactivity

If regular JavaScript doesn't have reactivity, how does Svelte know which variables should trigger screen updates?

Should we wrap every single variable in a heavy, complex JavaScript object like `new ReactiveVariable(0)` and call special methods like `count.set(1)` to change them? Other frameworks do this, but it makes writing simple math and logic very tedious.

We want to write normal JavaScript, like `count++`, but we need a way to tell the Svelte compiler, "Hey, this specific variable is special. Please add the secret reactivity code to it when you build my app." In Svelte 5, these special compiler symbols are called **runes**. Runes are just visual flags that tell the compiler to upgrade a normal variable into a reactive one.

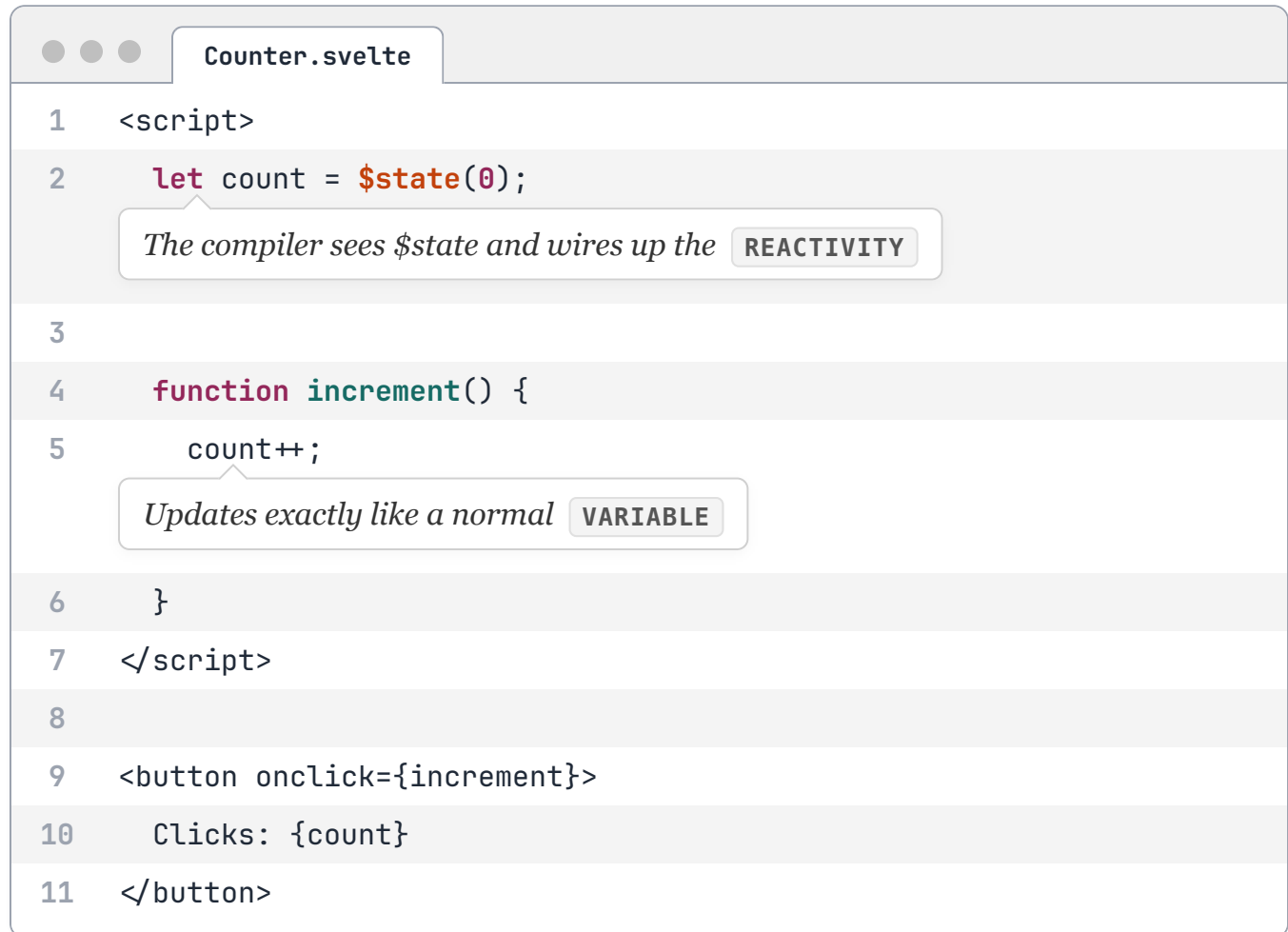
KEY TAKEAWAY

Runes are not JavaScript functions; they are instructions for the Svelte compiler.

Because runes are compiler instructions rather than runtime library functions, you don't even need to import `$state`. It acts as a built-in keyword in the Svelte language.

The `$state` Rune

To make a variable reactive, you initialize it with the `$state` rune.



```
1 <script>
2   let count = $state(0);
3
4   function increment() {
5     count++;
6   }
7 </script>
8
9 <button onclick={increment}>
10   Clicks: {count}
11 </button>
```

The compiler sees `$state` and wires up the **REACTIVITY**

Updates exactly like a normal **VARIABLE**

When the `increment` function runs and `count` increases, Svelte automatically detects the assignment (`=` , `++` , `+=`) and precisely updates only the part of the HTML where `{count}` is rendered.

Deep Reactivity

If you pass an array or an object into `$state`, Svelte makes it *deeply* reactive.



```
1 <script>
2   let todos = $state([ { text: 'Buy milk', done: false } ]);
3
4   function completeFirst() {
5     todos[0].done = true;
6   }
7 </script>
```

Triggers an update deep inside the **OBJECT**

Modifying a property deep inside the object, or calling methods like `todos.push()`, will automatically trigger the UI to update. Svelte wraps the object in a JavaScript Proxy behind the scenes, watching for any changes to its properties.

INTERVIEW TIP

To impress the interviewer: Contrast Svelte 5's `$state` with React's `useState`. Point out that Svelte's approach allows you to update variables directly (`count++`) rather than requiring a setter function (`setCount(count + 1)`), resulting in much cleaner, more natural JavaScript.

Summary

To create reactive variables in Svelte, you use the `$state` rune. This tells the compiler to automatically update the user interface whenever the variable is reassigned or modified. Because it's a compiler feature, you interact with the reactive data just like normal JavaScript variables, without needing special setter functions.

	SVELTE 5 (<code>\$state</code>) (Rune Reactivity)	REACT (<code>useState</code>) (Hook Reactivity)
Updating	Direct assignment (<code>count++</code>)	Setter function (<code>setCount(count + 1)</code>)
Deep Objects	Deeply reactive by default	Requires spreading/creating new objects
Mental Model	Compiler upgrades regular variables	Framework re-runs the entire function

Lecture 5: Handling User Events

INTERVIEW QUESTION

[> CORE](#)

How do you handle user events like clicks in Svelte?

You want a counter that goes up every time the user clicks a button. Where does the click handler go, and how does it connect to your state?

Interactive web applications rely on responding to user actions: clicking buttons, submitting forms, moving the mouse, or pressing keys.

The Problem with Manual Event Listeners

Imagine building a checkout page where a user clicks a "Buy Now" button to complete their purchase.

In vanilla JavaScript, to make that button do something, you have to query the DOM: `document.getElementById('buy-btn').addEventListener('click', buyItems)`. This is fragile. If you accidentally rename the ID in your HTML but forget to update the JavaScript, the connection is severed. Your user clicks "Buy Now", nothing happens, and you lose the sale. Furthermore, if you dynamically create and destroy buttons, you have to manually detach those event listeners to prevent catastrophic memory leaks. The sheer disconnect between the HTML element and its JavaScript behavior creates a massive surface area for bugs.

We need a way to bind the behavior *directly* to the HTML element where it is defined, so they are impossible to separate.

Event Attributes

To solve this, Svelte 5 allows you to pass JavaScript functions directly into standard HTML attributes like `onclick`, `oninput`, or `onkeydown`. This is known as using **event attributes**. By wiring the event listener directly in the template, Svelte guarantees it is safely attached when the element is created and perfectly cleaned up when the element is destroyed.

KEY TAKEAWAY

In Svelte 5, event listeners are just regular properties. They don't require special framework syntax.

Wiring a Click Handler

To handle a click event, you simply define a function in your `<script>` and assign it to the `onclick` attribute of your HTML element.



```
1  <script>
2    let items = $state(0);
3
4    function buyItem() {
5      items++;
6    }
7  </script>
8
9  <button onclick={buyItem}>
10    Buy Item ({items})
11  </button>
```

Updates the reactive state

The onclick attribute wires the button directly to the function

Notice the syntax: `onclick={buyItem}`. We pass the function reference directly inside the curly braces. We do *not* invoke the function like `onclick={buyItem()}` —doing so would run the function immediately when the page loads, instead of waiting for the user to click.

Accessing the Event Object

Sometimes you need details about the user's interaction, such as where they clicked on the screen or what key they pressed. When the browser triggers an event, it automatically passes an `event` object to your handler.



```
Tracker.svelte
1  <script>
2    function trackMouse(event) {
3      console.log(`Mouse is at X: ${event.clientX}`);
4    }
5  </script>
6
7  <div onmousemove={trackMouse}>
8    Move your mouse over me!
9  </div>
```

Accesses the native browser event

INTERVIEW TIP

To impress the interviewer: Mention that Svelte 5 changed event handling from Svelte 4's custom directive syntax (`on:click={handler}`) to standard HTML attributes (`onclick={handler}`). This change aligns Svelte closer to native browser standards and allows for much easier passing (spreading) of event handlers as props.

Summary

You handle user events in Svelte by assigning functions directly to HTML attributes like `onclick` . The function lives in your `<script>` block and can freely modify your `$state` variables. Svelte automatically manages attaching the native event listeners, ensuring your code remains tightly coupled to the UI without the fragility of manual DOM queries.

	MANUAL (Vanilla JS)	SVELTE 5 (<code>onclick</code> attribute)
Binding Method	<code>element.addEventListener('click', fn)</code>	<code>onclick={fn}</code> in template
Element Targeting	Fragile <code>id</code> or <code>class</code> queries	Bound directly to the HTML tag
Memory Management	Must manually <code>removeEventListener</code>	Automatically cleaned up by compiler

Lecture 6: Outputting Dynamic Values

INTERVIEW QUESTION

[> CORE](#)

How do you output dynamic values and set attributes in Svelte markup?

You have a variable in your script and want its value to appear inside a heading, or drive an element's class. How does markup read your script?

In modern web development, hardcoding static values into HTML is rarely enough. Your user interface needs to reflect the live data flowing through your application in real-time.

The Problem with Manual DOM Updates

Imagine you are building a shopping cart, and you need to display the current number of items. In vanilla JavaScript, you would define an empty placeholder in your HTML: `<span id="cart-count"></span>`. Then, in your JavaScript code, you write `document.getElementById('cart-count').innerText = count`. If the cart is empty, you might want to disable the checkout button by writing `document.getElementById('checkout-btn').disabled = true`.

You are constantly writing imperative code to reach across the boundary between HTML and JavaScript, manually gluing data into place. This is tedious to write, fragile to maintain, and easy to break if the DOM structure changes.

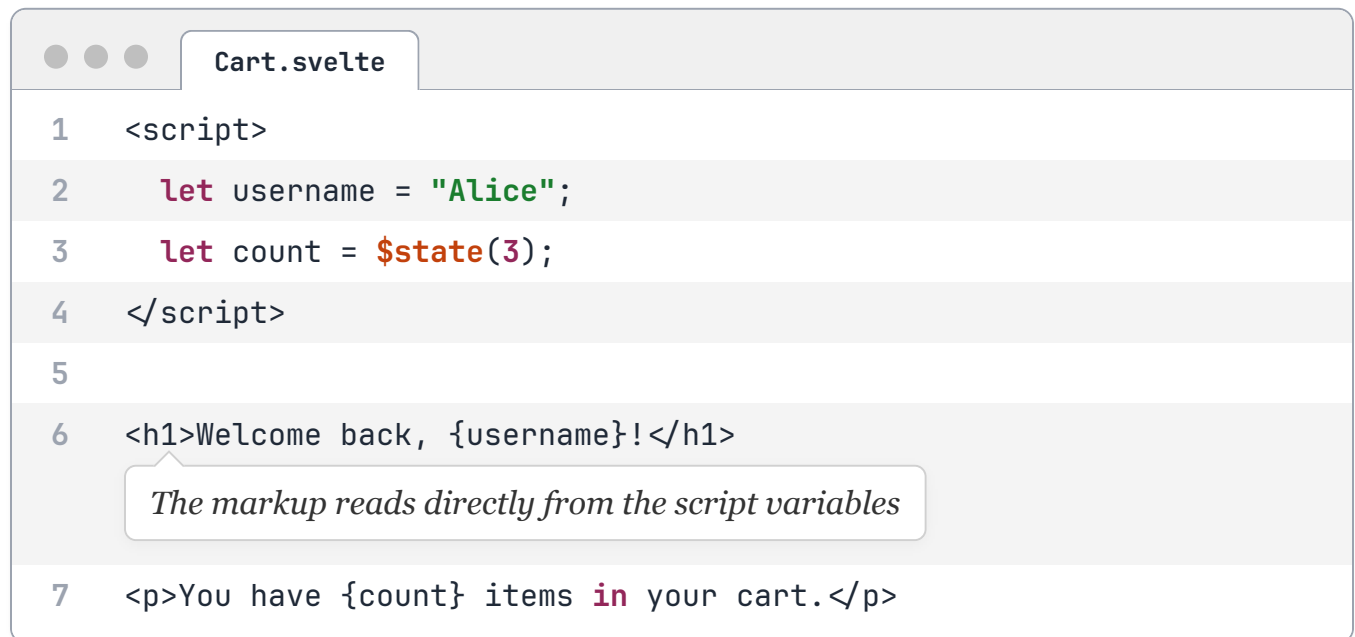
The Curly Braces Portal

Svelte solves this by allowing your markup to read directly from your script. By wrapping any JavaScript expression in **curly braces** (`{ }`), you open a portal into your `<script>` block right in the middle of your HTML.

Whenever the reactive value inside the braces changes, Svelte automatically updates the exact DOM node that relies on it.

Interpolating Text

To output a variable as text, just place the curly braces inside an HTML element. Svelte will evaluate the expression, convert the result to a string, and safely inject it as text (automatically escaping it to protect you from XSS injection attacks).



```
1 <script>
2   let username = "Alice";
3   let count = $state(3);
4 </script>
5
6 <h1>Welcome back, {username}!</h1>
7 <p>You have {count} items in your cart.</p>
```

The markup reads directly from the script variables

You can put any valid JavaScript expression inside the braces, not just variable names: `{count * 2}` , `{username.toUpperCase()}` , or `{count > 0 ? 'Full' : 'Empty'}` .

Dynamic Attributes

You can use the exact same curly brace syntax to set HTML attributes dynamically. Instead of a hardcoded string like `class="active"` , you pass the dynamic variable.

● ● ● Button.svelte

```
1 <script>
2   let isSubmitting = $state(false);
3   let themeColor = "blue";
4 </script>
5
6 <button disabled={isSubmitting} class={themeColor}>
  The disabled attribute and class are driven by variables
7   Checkout
8 </button>
```

KEY TAKEAWAY

If a boolean attribute (like `disabled` or `checked`) receives a truthy value, Svelte includes it. If it receives a falsy value, Svelte removes it completely from the DOM.

The Attribute Shorthand

Developers frequently name their variables identically to the attributes they want to control. For example, controlling an `id` with a variable named `id`, or `disabled` with a variable named `disabled`.

When the attribute name and the variable name match perfectly, Svelte provides a convenient shorthand: you can just write `{name}` instead of `name={name}`.



```
1 <script>
2   let disabled = $state(true);
3   let value = "Hello";
4 </script>
5
6 <input {disabled} {value} />
```

The shorthand `{disabled}` is identical to `disabled={disabled}`

INTERVIEW TIP

To impress the interviewer: Mention that unlike some frameworks that require special prefixes to bind attributes (like `v-bind:class` or `:class`), Svelte's curly braces work exactly the same way whether you are outputting text content or setting attribute values. It's a single, unified mental model.

Summary

To bridge the gap between your script and your markup, Svelte uses curly braces `{}` . This syntax evaluates JavaScript expressions and automatically updates the DOM when the underlying state changes. You can use curly braces to output text between HTML tags, or to dynamically set the values of HTML attributes. When an attribute and its controlling variable share the exact same name, Svelte offers a clean `{attribute}` shorthand.

	MANUAL (Vanilla JS)	SVELTE (Template Syntax)
Outputting Text	<code>element.innerText = value</code>	<code><h1>{value}</h1></code>
Setting Attributes	<code>element.disabled = true</code>	<code><button disabled={true}></code>
Matching Shorthand	N/A	<code><button {disabled}></code>

Lecture 7: Derived State

INTERVIEW QUESTION

[> CORE](#)

How do you compute a value that updates automatically when its inputs change?

You have a cart total that should recalculate whenever a price or quantity changes. Writing a function to recompute it by hand feels wrong.

State is the source of truth for your application. But often, the values you display in the UI aren't the raw state itself, but rather a *computation* based on that state.

The Problem with Manual Recalculation

Imagine you are building a checkout page with a list of items, and you need to display the grand total. If you create a standalone `$state(0)` variable for the total, you must manually update it every single time the cart changes.

When a user adds an item, you must call `updateTotal()`. When they remove an item, you must call `updateTotal()`. When they apply a 10 percent discount code, you must call `updateTotal()`. If another developer joins the team and adds a feature to change an item's quantity but forgets to call `updateTotal()`, the cart goes out of sync. Your user gets charged the wrong amount, resulting in a severe customer service nightmare.

The underlying problem is that the cart data and the total are fundamentally disconnected. The total *depends* on the cart, but your imperative code treats them as independent variables.

The `$derived` Rune

To fix this, Svelte 5 introduces the `$derived` rune. It allows you to declare a variable that is automatically calculated from other reactive variables.

● ● ● Checkout.svelte

```
1  <script>
2    let price = $state(20);
3    let quantity = $state(2);
4
5    let total = $derived(price * quantity);
6  </script>
7
8  <button onclick={() => quantity++}>
9    Add Item (Total: ${total})
10 </button>
```

Automatically stays in SYNC

When you declare `let total = $derived(price * quantity)`, you are telling the Svelte compiler to treat this value as entirely dependent on `price` and `quantity`.

You never have to manually update `total`. Whenever `price` or `quantity` changes, Svelte knows it needs to recalculate `total` before updating the DOM.

KEY TAKEAWAY

If `$state` is the source of truth, `$derived` is the automatic reflection of that truth.

Complex Derivations with `$derived.by`

Sometimes your computation is too complex to fit on a single line, like iterating over an array to sum up prices. For these cases, you can use `$derived.by`, which accepts a function block.



```
1  <script>
2    let cart = $state([10, 20, 30]);
3
4    let grandTotal = $derived.by(() => {
5      let sum = 0;
6      for (const item of cart) {
7        sum += item;
8      }
9      return sum;
10   });
11 </script>
```

*Iterates and sums up the **REACTIVE** array*

Push-Pull Reactivity

Svelte uses a highly optimized system called *push-pull reactivity*. When a `$state` variable changes, Svelte immediately pushes a notification to everything that depends on it, marking those derived values as dirty. However, it doesn't actually do the mathematical recalculation yet. It waits until the exact moment the UI tries to pull or read the value. This ensures Svelte never wastes CPU cycles calculating values that are never displayed on screen.

Furthermore, if the newly calculated value happens to be identical to the old value, Svelte halts the update right there, preventing unnecessary DOM updates.

INTERVIEW TIP

To impress the interviewer: Mention that `$derived` replaces Svelte 4's reactive statements (the `$:` label). While the old syntax felt like magic, the new `$derived` rune behaves like a standard JavaScript variable declaration, making it much easier to reason about scope and refactor into separate `.svelte.js` files.

Summary

When a value in your UI depends on other reactive data, never update it manually. Instead, use the `$derived` rune (or `$derived.by` for complex logic). Svelte's compiler automatically tracks the dependencies and ensures the derived value is recalculated exactly when needed. This guarantees your UI is never out of sync with your underlying state, completely eliminating a massive class of bugs.

	MANUAL (Vanilla JS)	SVELTE 5 (<code>\$derived</code>)
Declaration	<code>let total = 0;</code>	<code>let total = \$derived(p * q);</code>
Updating	Must call <code>update()</code> manually	Automatically tracked by compiler
Performance	Runs whenever you explicitly call it	Optimized push-pull recalculation

Lecture 8: Child Components

INTERVIEW QUESTION

[> CORE](#)

How do you use one component inside another?

Your `App.svelte` is getting crowded. How do you split it into smaller pieces and reference them?

As your web application grows, putting everything into a single file quickly becomes impossible to manage.

The Problem with Monolithic Files

Imagine you are building a dashboard application, and your `App.svelte` file has grown to 1,500 lines of code. It contains the logic for a top navigation bar, a massive collapsible sidebar, a complex interactive data grid, and a footer.

Whenever you need to fix a tiny bug in the sidebar, you have to scroll past hundreds of lines of unrelated data grid logic, desperately searching for the exact `<div>` you need to change. Worse, because all the variables and styles are trapped in the same global scope of that file, a teammate might accidentally change a variable meant for the header and inadvertently break the footer. Your codebase has become a tangled, fragile monolith.

Splitting into Child Components

To solve this, Svelte allows you to break your user interface into smaller, entirely self-contained building blocks called **components**.

Instead of one giant file, you cut the sidebar logic out and paste it into its own file named `Sidebar.svelte`. You do the same for the header in `Header.svelte`. Each component encapsulates its own HTML markup, its own CSS styling, and its own JavaScript logic.

Importing and Rendering

Once you have separated a piece of your interface into its own `.svelte` file, you need a way to bring it back into the main application.

You do this in two simple steps:

1. **Import the file** in your `<script>` tag, just like a standard JavaScript module.
2. **Render it** in your markup as if it were a custom HTML tag.

Svelte components must always be Capitalized when used in markup. This is how the Svelte compiler distinguishes your custom `<Sidebar>` component from a standard lowercase HTML `<div>` or `<span>` .

```
1  <script>
2    import Header from './Header.svelte';
3    import Sidebar from './Sidebar.svelte';
4  </script>
5
6  <div class="layout">
7    <Header />
8
9    <main>
10      <Sidebar />
11      <p>Main dashboard content goes here.</p>
12    </main>
13  </div>
```

Import the child component file

Render it like a custom HTML tag

The Component Tree

When you use one component inside another, you are creating a parent-child relationship. In the example above,

`App.svelte` is the **parent component**, while `Header` and `Sidebar` are its **child components**.

These child components can, in turn, import and render their own children (like a `<NavButton>` inside the `<Sidebar>`), creating a deeply nested Component Tree that perfectly mirrors the visual hierarchy of your application.

KEY TAKEAWAY

Each instance of a component is completely independent. If you render `<Sidebar />` twice on the same page, each one will get its own separate, isolated copy of any internal `$state` variables.

Summary

To prevent your files from becoming unmaintainable monoliths, you should split discrete parts of your UI into separate `.svelte` files. To use these files, you `import` them into the parent component's script block, and then render them in the markup using an uppercase tag. This encapsulation allows you to build complex, scalable applications out of small, reliable, and reusable building blocks.

	HTML TAGS (Native)	SVELTE COMPONENTS (Custom)
Naming	Lowercase (<code><div></code> , <code></code>)	Uppercase (<code><Widget></code> , <code><Sidebar></code>)
Source	Built into the browser natively	Imported from a <code>.svelte</code> file
Behavior	Standard DOM elements	Encapsulated custom logic and styling

Lecture 9: Component Props

INTERVIEW QUESTION

[> CORE](#)

How do you pass data from a parent component into a child?

You've built a `Button` component, but every instance looks identical. How do you give each one its own label?

Components allow you to encapsulate logic and styling, but a component is only truly useful if it is reusable. If every instance of a component is perfectly identical, its usefulness is severely limited.

The Problem with Hardcoded Components

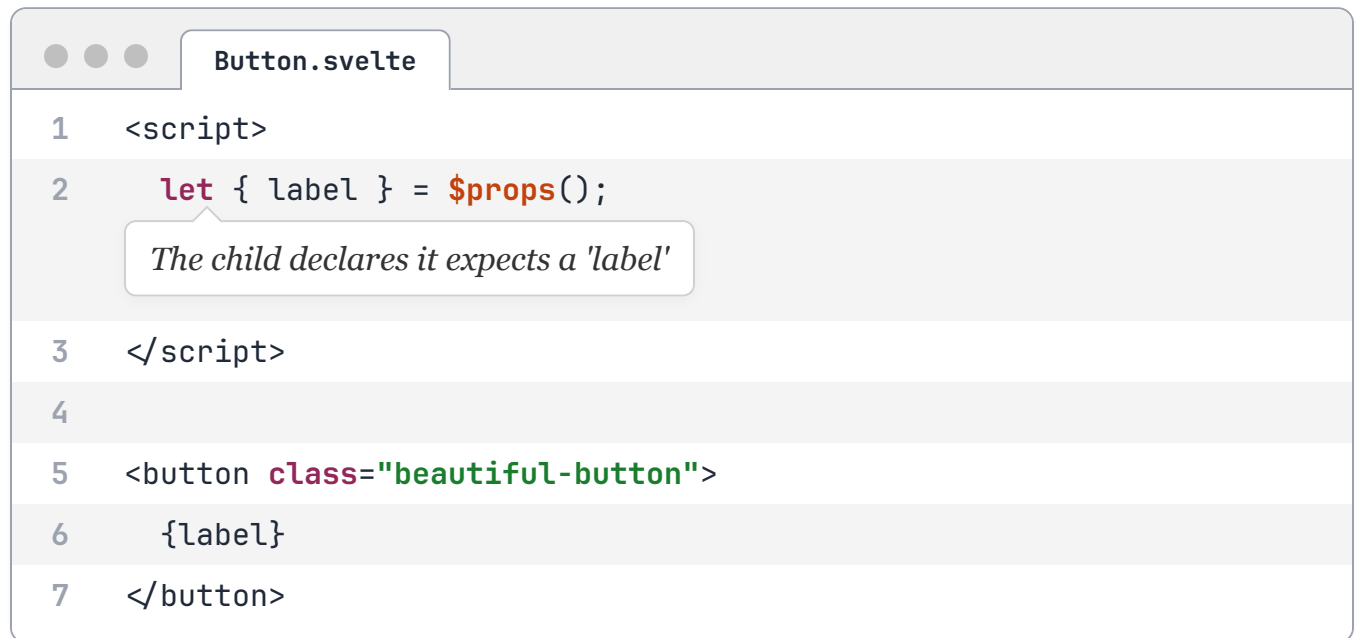
Imagine you extract a beautiful, perfectly styled `<Button>` into its own file. It has the right drop shadow, the right hover state, and the right typography. But inside `Button.svelte`, you hardcoded the text `Click Me`.

When you import that button into your dashboard, you need one button that says "Save", one that says "Delete", and one that says "Submit". Because the text is hardcoded inside the child, every single button says "Click Me". To fix this without props, you would have to duplicate the entire file into `SaveButton.svelte`, `DeleteButton.svelte`, and `SubmitButton.svelte`. This completely destroys the point of reusable components.

The `$props` Rune

To make components flexible, Svelte 5 provides the `$props` rune. Props (short for properties) act as the inputs to your component. They allow a parent component to pass data down into a child component.

Inside the child component, you declare that you expect to receive data by destructuring the `$props()` rune.



```
1 <script>
2   let { label } = $props();
3 </script>
4
5 <button class="beautiful-button">
6   {label}
7 </button>
```

The child declares it expects a 'label'

Passing Props from the Parent

Now that the child component is ready to receive a `label`, the parent component can pass that data in. You pass props to a Svelte component exactly the same way you pass attributes to a standard HTML element.



```
1  <script>
2    import Button from './Button.svelte';
3  </script>
4
5  <div class="actions">
6    <Button label="Save" />
7    <Button label="Delete" />
8  </div>
```

The parent passes a static string

Props are not limited to static strings. You can pass reactive `$state` variables, numbers, objects, or even arrays by using curly braces. Because props are tied directly into Svelte's reactivity system, if the parent's state changes, the child automatically updates.

● ● ● Dashboard.svelte

```
1  <script>
2    import Button from './Button.svelte';
3    let dynamicText = $state("Submit");
4  </script>
5
6  <Button label={dynamicText} />
```

The parent passes a reactive variable

KEY TAKEAWAY

Props flow strictly downwards: from parent to child. The parent owns the data, and the child merely receives it to render the UI.

Summary

To make components reusable, they need to accept dynamic inputs called props. In Svelte 5, the child component declares its expected inputs using `let { propName } = $props()` . The parent component passes data into those props using standard HTML attribute syntax. This downward flow of data allows you to build a single, highly flexible component (like a generic `<Button>`) and reuse it in dozens of different contexts just by changing the data you feed it.

	HTML ATTRIBUTES	COMPONENT PROPS
Recipient	Native DOM elements (<code><input></code>)	Svelte components (<code><Button></code>)
Declaration	Built into the browser	Declared via <code>let { ... } = \$props()</code>
Data Types	Mostly strings and booleans	Any JavaScript type (Objects, Arrays)

Lecture 10: Defaults and Rest Props

INTERVIEW QUESTION

[> CORE](#)

How do you give a prop a default value and gather the rest?

Some props are optional, and sometimes you want to forward whatever the parent passed straight to a DOM element. How do defaults and "the rest of the props" work?

When building robust components, you often want to provide sensible defaults for optional settings, while still allowing the parent to customize them. Furthermore, you need a way to gracefully handle native HTML attributes without manually declaring every single one.

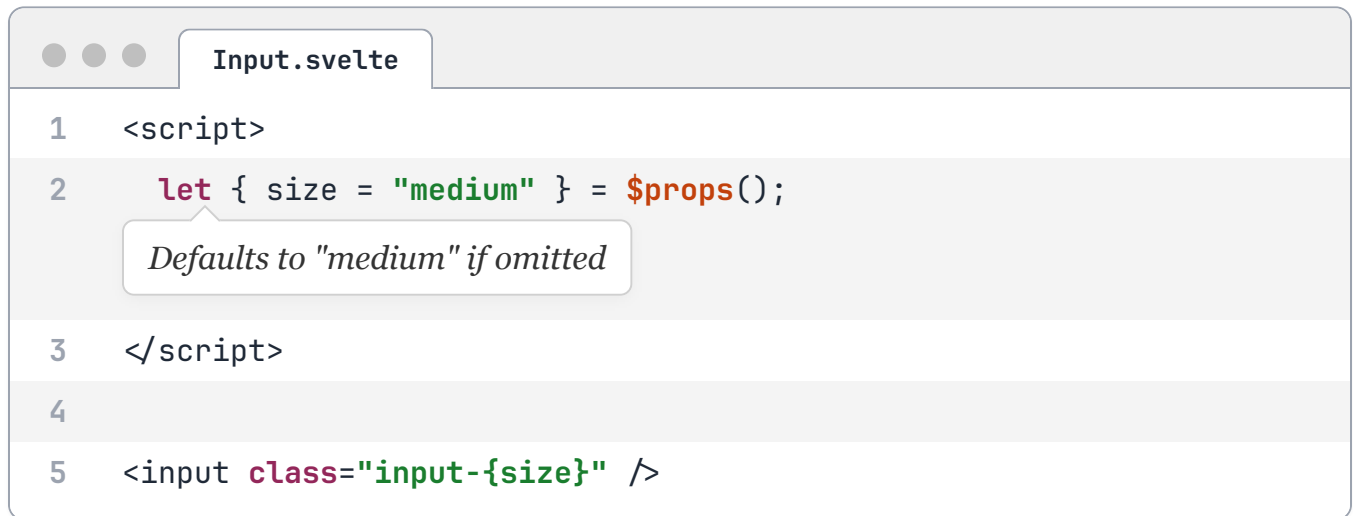
The Exhaustive Declaration Problem

Imagine you are building an `Input.svelte` component. You decide to offer a `size` prop to control its visual scale, but you want it to default to "medium" so developers don't have to specify it every time.

More problematically, a standard HTML `<input>` tag supports dozens of attributes: `disabled`, `readonly`, `placeholder`, `maxlength`, `pattern`, `type`, and more. If you want your custom component to support all of these standard features, manually defining every single one inside your `$props()` block is a massive waste of time. If you forget to declare `pattern`, your component will silently swallow the attribute, breaking form validation and infuriating your teammates who expect your `<Input>` to behave like a native browser element.

Default Values

Because Svelte 5's `$props` rune uses standard JavaScript destructuring syntax, you can assign default fallback values directly in the declaration. If the parent component does not provide the prop, Svelte falls back to the default value automatically.



```
1 <script>
2   let { size = "medium" } = $props();
3 </script>
4
5 <input class="input-{size}" />
```

Now, the parent can render `<Input />` for a standard input, or `<Input size="large" />` for an oversized one.

The Rest Props Pattern

To solve the issue of forwarding native HTML attributes, you can use the JavaScript **rest property** syntax (`...`). By putting `...rest` at the end of your destructuring block, Svelte gathers all the props that you *didn't* explicitly name and bundles them into a single object.

You can then use Svelte's spread syntax (`{...rest}`) in your markup to apply all of those gathered attributes directly onto a DOM element in one fell swoop.

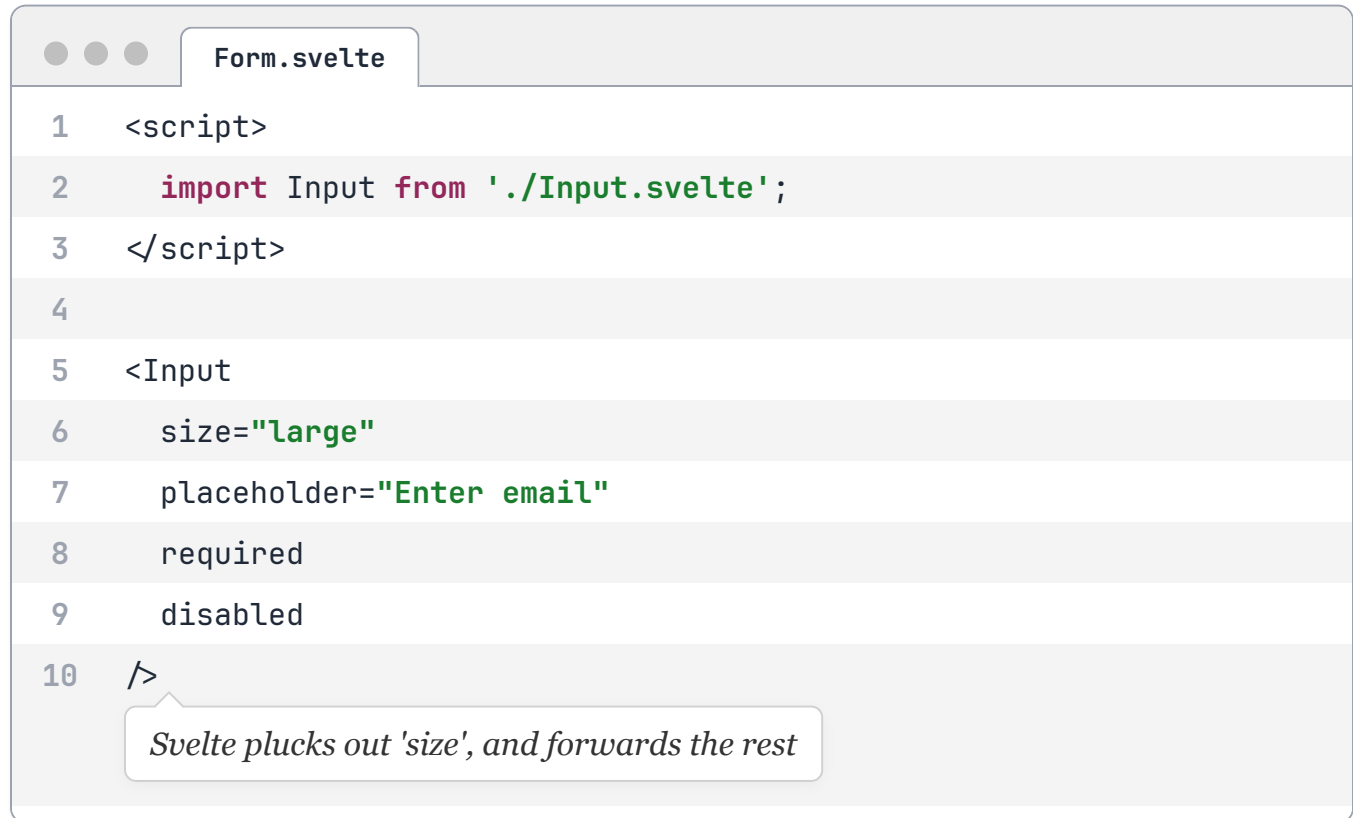
```
1  <script>
2    let {
3      size = "medium",
4      ...rest
5    } = $props();
6  </script>
7
8  <input class="input-{size}" {...rest} />
```

Gathers everything else into an object named 'rest'

Spreads all undeclared attributes onto the native input

Forwarding in Action

When a parent uses this component, they can pass arbitrary HTML attributes without the child component needing to know about them explicitly:



```
1  <script>
2    import Input from './Input.svelte';
3  </script>
4
5  <Input
6    size="large"
7    placeholder="Enter email"
8    required
9    disabled
10 />
```

Svelte plucks out 'size', and forwards the rest

In this scenario, `size` is extracted and used for the CSS class. The `placeholder`, `required`, and `disabled` props are scooped up by the `...rest` object and seamlessly applied directly to the underlying native `<input>` tag.

INTERVIEW TIP

To impress the interviewer: Note that the order of the spread attribute matters. If you write `<input {...rest} class="my-class">`, your static class will overwrite any `class` passed in via `...rest`. Conversely, writing `<input class="my-class" {...rest}>` allows the parent component to override your internal classes.

Summary

Svelte 5 leans entirely on standard JavaScript language features to handle complex prop scenarios. You can set optional fallback values using standard destructuring assignment (`= "default"`). To build flexible wrapper components that mimic native HTML elements, you use the rest operator (`...rest`) to gather all undeclared props, and then use the spread operator (`{...rest}`) in your markup to forward those attributes directly to the underlying DOM node.

	DEFAULTS	REST PROPS
Syntax	<pre>let { size = 'md' } = \$props()</pre>	<pre>let { size, ...rest } = \$props()</pre>
Markup Usage	<pre>{size}</pre>	<pre><input {...rest} /></pre>
Primary Use Case	Optional component configurations	Forwarding native HTML attributes

Lecture 11: Callback Props

INTERVIEW QUESTION

> CORE

How does a child component tell its parent that something happened?

Your `Button` is clicked, but the parent needs to know. How does the child send a message up the tree without a global store?

Data in Svelte flows strictly downwards via props. But applications are interactive; when a user takes an action inside a nested component, the broader application needs to react.

The Upward Communication Problem

Imagine you are building an e-commerce site. You have an `App.svelte` parent component that holds the shopping cart state. Inside it, you render a list of `<ProductCard>` child components, each featuring an "Add to Cart" button.

When a user clicks that button, the click event fires completely inside the child component. However, the shopping cart data lives up in the parent. If the child has no way to alert the parent, the user clicks the button repeatedly, the cart remains empty, and the user leaves the site in frustration. You need a reliable way for the child to shout "Hey, this item was clicked!" up the component tree.

Functions as Props

In Svelte 5, the solution is incredibly simple: functions are just JavaScript values, which means you can pass them as props.

The parent component defines a function that updates its own state. It then passes that exact function down to the child component as a prop (often called a **callback**).

● ● ● App.svelte

```
1  <script>
2    import ProductCard from './ProductCard.svelte';
3
4    let cartCount = $state(0);
5
6    function handleAdd() {
7      cartCount++;
8    }
9  </script>
10
11  <p>Items in cart: {cartCount}</p>
12
13  <ProductCard onadd={handleAdd} />
```

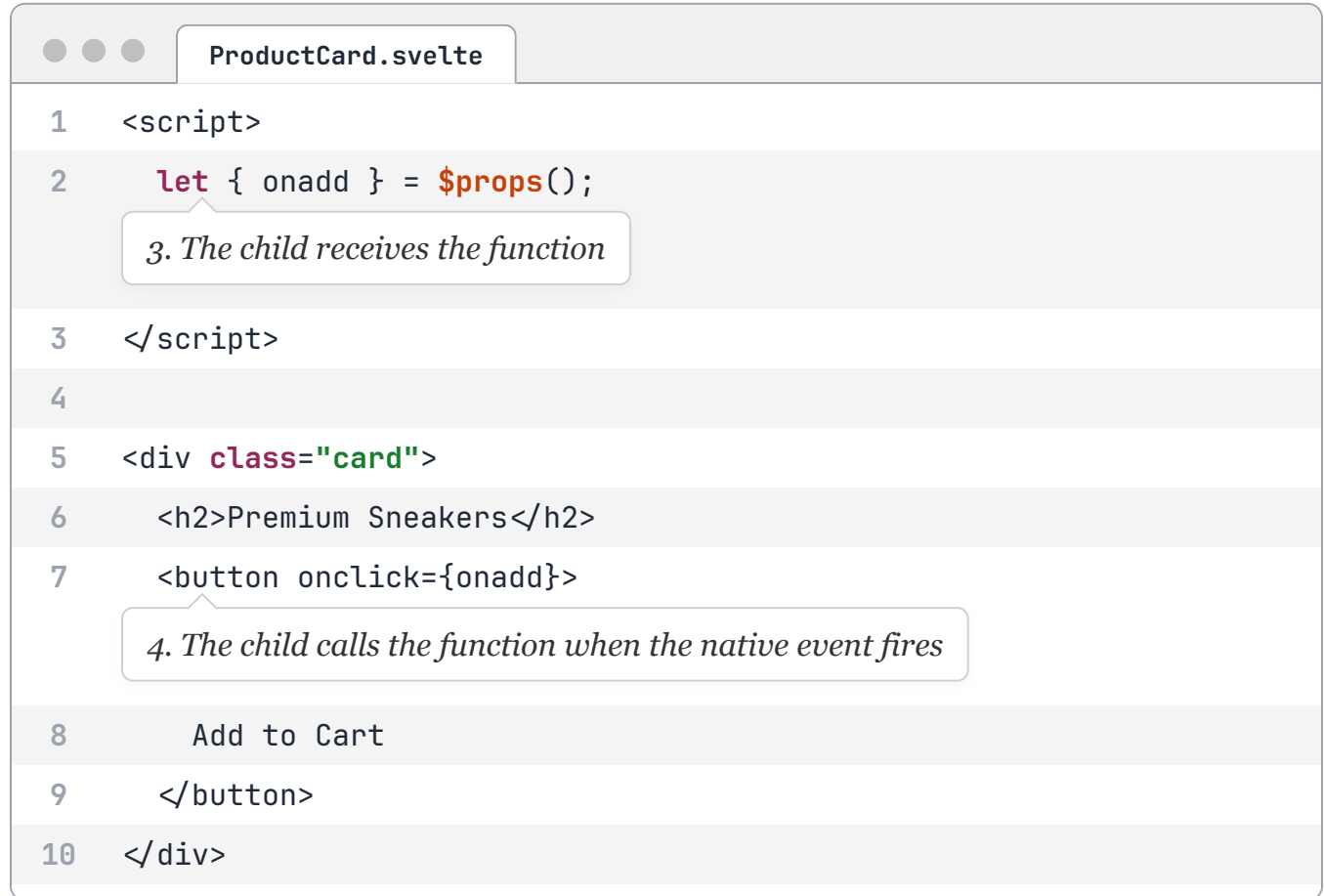
1. The parent defines the function

2. The parent passes the function down as a prop

Triggering the Callback

Inside the child component, it declares the callback function in its `$props()` block just like any other data. When the user interacts with the UI, the child simply executes the function.

Because the function was defined in the parent's scope, executing it updates the parent's state.



```
1  <script>
2    let { onadd } = $props();
3  </script>
4
5  <div class="card">
6    <h2>Premium Sneakers</h2>
7    <button onclick={onadd}>
8      Add to Cart
9    </button>
10 </div>
```

3. The child receives the function

4. The child calls the function when the native event fires

Passing Data Upwards

Callback functions aren't just for signaling that an event happened; they can also carry data. If the child needs to tell the parent exactly *what* was clicked, it can pass arguments into the callback function.

● ● ● ProductCard.svelte

```
1  <script>
2    let { id, onadd } = $props();
3  </script>
4
5  <button onclick={() => onadd(id)}>
    Add to Cart
7  </button>
```

The child passes its own data back up to the parent

The parent function `function handleAdd(productId)` will receive the ID directly from the child.

INTERVIEW TIP

To impress the interviewer: If they ask about `createEventDispatcher`, confidently explain that it was deprecated in Svelte 5. Callback props have replaced custom event dispatchers because they are simpler, faster, and provide vastly superior type-safety natively in TypeScript without requiring custom generic types.

Summary

While data flows down, events flow up. To allow a child component to communicate with its parent in Svelte 5, the parent passes a function down as a prop. The child executes this callback function when an action occurs, optionally passing internal data as arguments. Because the function is bound to the parent's scope, it can safely modify the parent's state, bridging the gap between nested components seamlessly.

	SVELTE 4 (Legacy)	SVELTE 5 (Modern)
Mechanism	<code>createEventDispatcher()</code>	Callback functions via <code>\$props()</code>
Parent Syntax	<code><Component on:click={fn} /></code>	<code><Component onclick={fn} /></code>
Type Safety	Complex custom generic interfaces	Native TypeScript function types

Lecture 12: Conditional Rendering

INTERVIEW QUESTION

[> CORE](#)

How do you show, hide, or branch parts of the UI based on state?

You only want to show the login form when the user is signed out, and the dashboard when they're signed in. How do you conditionally render markup?

A static web page looks the same to every visitor. A web application, however, must adapt its interface based on the user's state, permissions, or actions.

The CSS Hiding Vulnerability

Imagine you are building a banking application. Users must log in before they can view their account balance. In older vanilla JavaScript patterns, developers would often render both the login form and the private dashboard into the HTML, and then use CSS (`display: none`) to hide the dashboard until the user authenticated.

This approach is catastrophic for security and performance. Even if the dashboard is visually hidden, a malicious user can simply open their browser's Developer Tools, inspect the DOM, and see the entire private dashboard HTML structure sitting right there. Furthermore, the browser is forced to download and parse heavy dashboard components that the user might never even see. You need a way to physically prevent HTML from existing in the document until a specific condition is met.

The `{#if}` Block

Svelte solves this using the `{#if}` logic block. It allows you to wrap portions of your markup in conditional statements.

```
App.svelte
1  <script>
2    let loggedIn = $state(false);
3  </script>
4
5  {#if loggedIn}
6    <h1>Welcome to your Dashboard!</h1>
7  {/if}
```

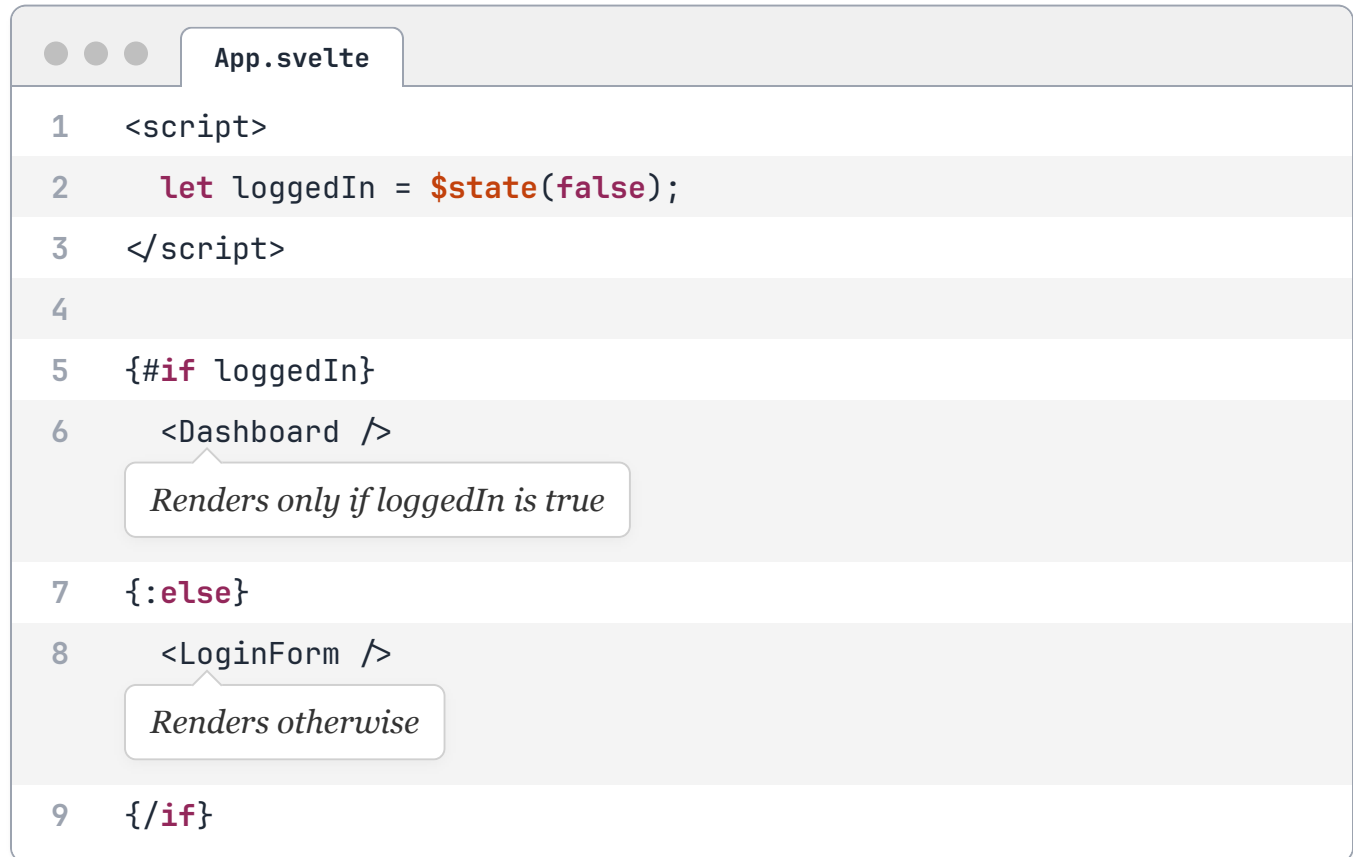
Unlike CSS toggles, Svelte's `{#if}` block mounts and unmounts elements. If `loggedIn` is false, the `<h1>` does not exist in the DOM at all. It is a completely impenetrable void. When `loggedIn` becomes true, Svelte instantly constructs the HTML and injects it into the page.

KEY TAKEAWAY

Logic blocks in Svelte start with a `#`, can have intermediate branching steps starting with a `:`, and always close with a `/`.

Branching with `{:else}`

Often, you don't just want to show or hide one thing; you want to toggle between two distinct interfaces. You can chain an `{:else}` block to handle the fallback state.



```
1  <script>
2    let loggedIn = $state(false);
3  </script>
4
5  {#if loggedIn}
6    <Dashboard />
7  {:else}
8    <LoginForm />
9  {/if}
```

Renders only if loggedIn is true

Renders otherwise

Complex Logic with {:else if}

For scenarios with more than two possible states, you can chain multiple conditions together using `{:else if}`. The first condition that evaluates to true will render its block, and Svelte will ignore the rest.



The screenshot shows a code editor window titled "TrafficLight.svelte". The code is as follows:

```
1  <script>
2    let status = $state('warning');
3  </script>
4
5  {#if status === 'danger'}
6    <p class="red">Stop immediately!</p>
7  {:else if status === 'warning'}
8    <p class="yellow">Proceed with caution.</p>
9  {:else}
10   <p class="green">All clear to go.</p>
11  {/if}
```

Summary

To conditionally render parts of your UI, Svelte uses the `{#if}` , `{:else if}` , and `{:else}` logic blocks. Crucially, these blocks do not merely visually hide elements using CSS; they physically construct and destroy DOM nodes based on reactive state. This ensures your application remains secure, lightweight, and perfectly synchronized with its underlying logic.

	CSS HIDING (Legacy)	SVELTE <code>{#if}</code> (Modern)
DOM Presence	Always in the DOM	Physically added/ removed
Security	Vulnerable to DOM inspection	Secure (HTML does not exist)
Syntax	<code><div class={isHidden ? 'hide' : ''}></code>	<code>{#if isVisible} <div> {/if}</code>

Lecture 13: Iterating with Each Blocks

INTERVIEW QUESTION

[> CORE](#)

How do you render a list of items from an array, and why does each need a key?

You have ten users and want a row for each. When you sort the list, the rows lose their state and input values jump around. What went wrong?

Most web applications display lists of data: a feed of social media posts, a grid of products, or a table of users.

The Identity Crisis

Imagine you are building a banking dashboard with a list of "Pending Transactions". Each transaction row has a text input where the user can type a memo before approving it. The user types "Rent" into the first input, then clicks a button to sort the transactions by date.

Suddenly, the "Rent" note jumps to a completely different transaction, attaching itself to a \$5 coffee bill! What went wrong?

Without strict instructions, a rendering engine takes the path of least resistance. When the array sorted, it didn't actually move the HTML rows around. Instead, it left the HTML elements exactly where they were, and just updated the text inside them to match the new array order. But the text you typed into the `<input>` wasn't part of the array data; it was internal DOM state. Because the row itself never moved, the input text stayed at the top of the list, while the transaction data shifted beneath it.

The `{#each}` Block

To render a list of items in Svelte, you use the `{#each}` block. It takes an array and loops through it, rendering a block of HTML for every item.

To prevent the catastrophic identity crisis described above, Svelte allows you to define a **key** by providing a unique identifier in parentheses at the end of the declaration.

```
Transactions.svelte
1  <script>
2    let transactions = $state([
3      { id: 'tx_1', amount: 50, merchant: 'Coffee Shop' },
4      { id: 'tx_2', amount: 1200, merchant: 'Landlord' }
5    ]);
6  </script>
7
8  {#each transactions as tx (tx.id)}
   The (tx.id) is the unique key mapping data to the DOM
9    <div class="row">
10      <p>{tx.merchant}: ${tx.amount}</p>
11      <input placeholder="Add memo..." />
12    </div>
13  {/each}
```

Why Keys are Mandatory

When you provide `(tx.id)` as the key, you are making a binding contract with the Svelte compiler. You are telling it: *"This specific piece of data is permanently married to this specific HTML `<div>`."*

Now, when the array is sorted, Svelte will not cheat by just updating the text. It will grab the exact DOM node tied to `tx_2` and physically move it down the page. Because the entire HTML element moves, any internal state (like the text the user typed into the memo input, or CSS animations currently playing on that node) moves perfectly with it.

KEY TAKEAWAY

Always use a unique, stable database ID for your key.
Never use the array index `( index )` as a key if the list can be reordered, because the index of an item changes when it moves, defeating the entire purpose of the key.

Summary

To render arrays of data, wrap your markup in an `{#each array as item}` logic block. To ensure the DOM updates safely and predictably when the array changes, you must provide a unique identifier for every item using the `(key)` syntax. This key locks the data to a specific DOM node, ensuring that sorting, filtering, or deleting items physically moves the correct HTML elements, preserving any internal DOM state like focus or user input.

	NO KEY (Unsafe)	WITH KEY (Safe)
Syntax	<code>{#each items as item}</code>	<code>{#each items as item (item.id)}</code>
Sorting Behavior	Updates text in place	Physically moves DOM nodes
DOM State (Inputs)	Corrupts and jumps around	Moves safely with the item

Lecture 14: Binding Form Inputs

INTERVIEW QUESTION

[> CORE](#)

How do you bind form inputs to state?

Text inputs are easy, but checkboxes are booleans, radios pick one of many, and dropdowns can be multi-select. How do you bind each to state?

Building web forms in vanilla JavaScript is incredibly tedious. For every single input, you have to write an event listener to capture the user's keystrokes, and then write a separate function to push your data back into the input if it changes programmatically.

The Problem with One-Way Data Flow

Imagine you are building a complex user profile form with a text input for the username, a checkbox to accept terms, and a dropdown for their country.

If you just use `{value}` to set the input's value, data only flows in one direction: from your script to the DOM. When the user types into the input, the DOM changes, but your `$state` variable has no idea. To fix this, you have to attach an `oninput` handler to every single element to manually capture `event.target.value` and update your variables. This is boilerplate-heavy and quickly becomes unmanageable for large forms.

Two-Way Binding with `bind:`

Svelte solves this elegantly with the `bind:` directive. When you bind an input to a reactive variable, you establish a two-way connection. If the variable changes, the input updates. If the user types into the input, the variable updates automatically.



The image shows a code editor window titled "Profile.svelte". The code is as follows:

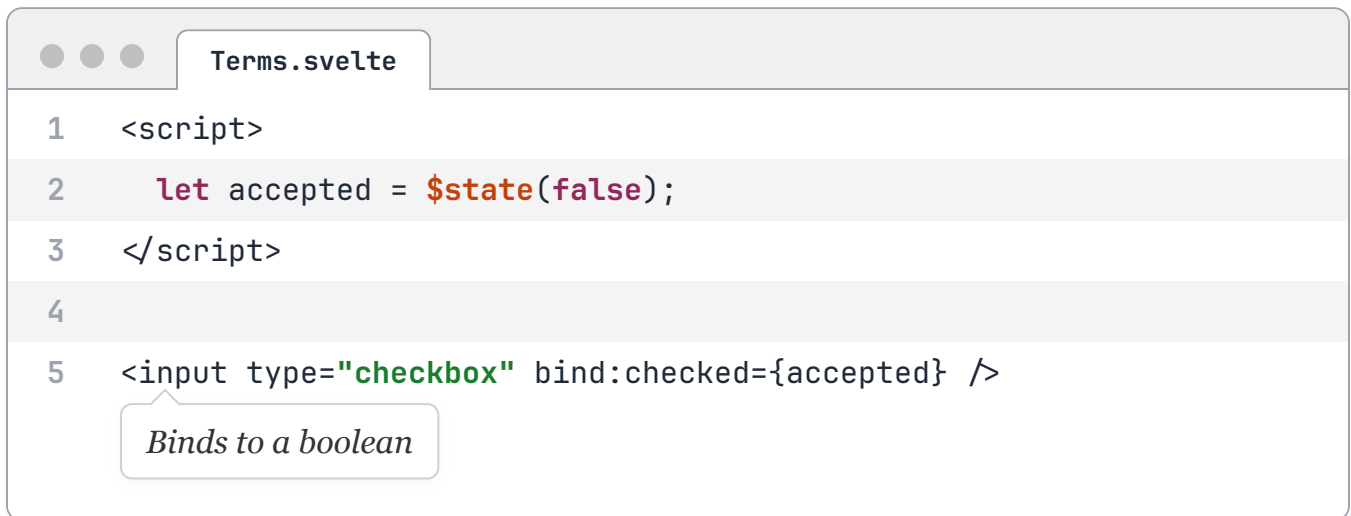
```
1  <script>
2    let username = $state("");
3  </script>
4
5  <input type="text" bind:value={username} />
6  <p>Hello, {username}</p>
```

A callout box points to the `bind:value={username}` attribute in line 5, containing the text *Two-way connection*.

Handling Different Input Types

The true power of Svelte's `bind:` directive is that it is smart enough to handle the idiosyncrasies of different HTML input types.

1. Checkboxes A checkbox doesn't have a string value; it is either checked or unchecked. You bind to its `checked` property using a boolean.



The screenshot shows a code editor window titled "Terms.svelte". It contains the following code:

```
1 <script>
2   let accepted = $state(false);
3 </script>
4
5 <input type="checkbox" bind:checked={accepted} />
```

A callout box points to the `bind:checked` attribute in line 5, containing the text "Binds to a boolean".

2. Radio Buttons Radio buttons work in groups where only one can be selected at a time. By binding all radios in a group to the same variable using `bind:group`, Svelte handles the mutual exclusivity for you.



The screenshot shows a code editor window titled "Survey.svelte". It contains the following code:

```
1 <script>
2   let difficulty = $state("easy");
3 </script>
4
5 <input type="radio" bind:group={difficulty} value="easy" />
6 <input type="radio" bind:group={difficulty} value="hard" />
```

3. Select Dropdowns For a standard `<select>` , you just bind to its `value` . For a `multiple` select, you bind to an array of values.

Country.svelte

```
1 <script>
2   let country = $state("US");
3 </script>
4
5 <select bind:value={country}>
  Value perfectly matches the selected option
6   <option value="US">United States</option>
7   <option value="UK">United Kingdom</option>
8 </select>
```

KEY TAKEAWAY

Numeric inputs (`type="number"` or `type="range"`) are notoriously annoying in JavaScript because `event.target.value` is always a string. Svelte automatically handles type coercion for you: if you `bind:value` on a number input, your variable will be a true number, not a string.

Summary

Forms require two-way data flow to stay synchronized with your application state. Instead of writing manual event listeners, use Svelte's `bind:` directive. It handles text inputs (`bind:value`), checkboxes (`bind:checked`), radio groups (`bind:group`), and select dropdowns natively, completely abstracting away the tedious boilerplate of vanilla DOM events.

	INPUT TYPE	SVELTE DIRECTIVE
Text & Numbers	<code><input type="text"></code>	<code>bind:value</code>
Checkboxes	<code><input type="checkbox"></code>	<code>bind:checked</code>
Radio Buttons	<code><input type="radio"></code>	<code>bind:group</code>

Lecture 15: Side Effects

INTERVIEW QUESTION

> CORE

How do you run a side effect when something reactive changes, and how do you clean it up?

You need to save a value to localStorage every time it changes, and remove an event listener when the effect re-runs or the component dies. Where does that logic live?

While `$derived` is used to calculate new data for your UI, sometimes you need to step outside of the UI and interact with the external world. These external interactions are called side effects.

The Problem with Unmanaged Effects

Imagine you are building a document editor that automatically saves the user's progress to the browser's `localStorage` every time they type. If you just put

```
localStorage.setItem('doc', text)
```

floating loose in your `<script>` block, it will only run exactly once when the component is first created. When the user types, the `text` variable updates, but the loose code never runs again.

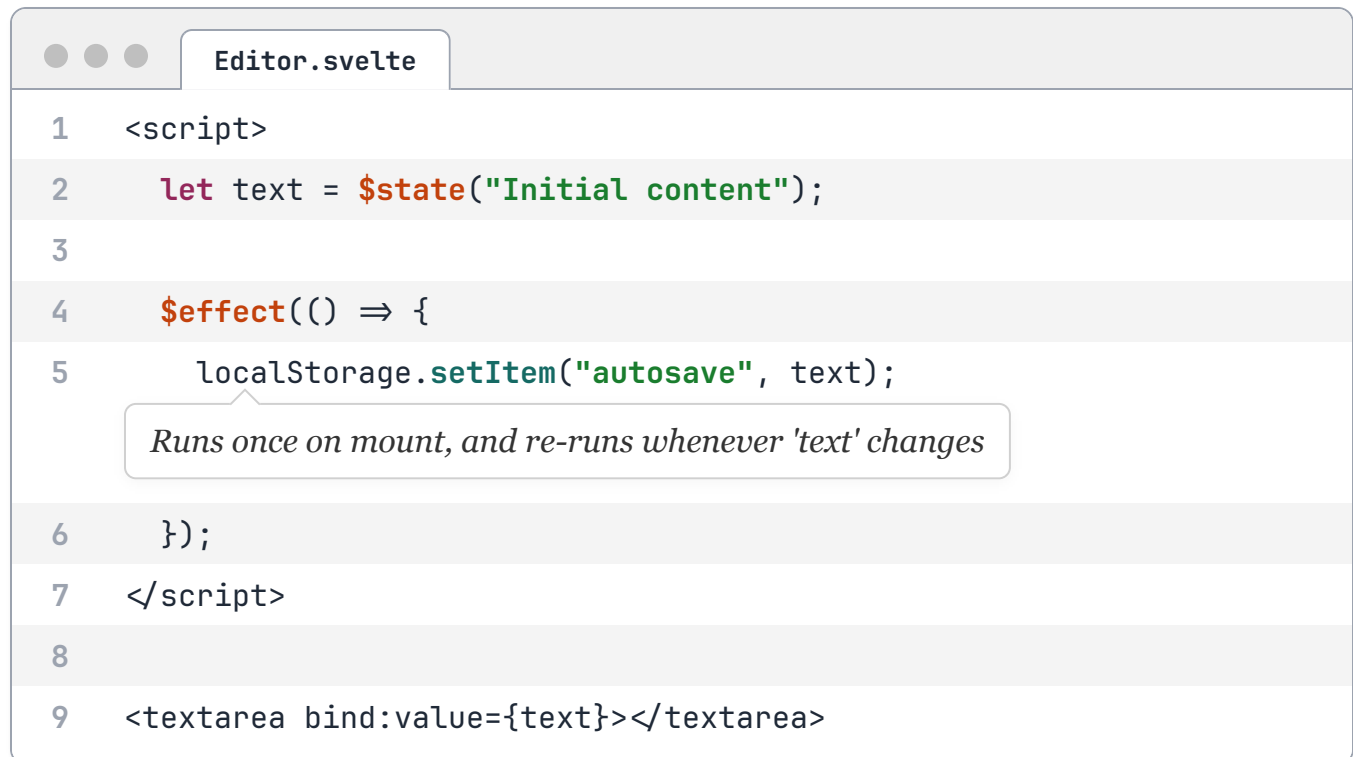
Conversely, if you manually add a

```
window.addEventListener('resize', updateLayout)
```

inside a component but forget to remove it when the user navigates away, you have created a memory leak. Every time the user visits that page, another permanent event listener is added, eventually grinding the browser to a halt. You need a safe, reactive wrapper for external code.

The `$effect` Rune

To solve this, Svelte 5 provides the `$effect` rune. It runs a block of code after the component has mounted to the DOM, and automatically re-runs that code whenever any reactive state inside it changes.



```
1  <script>
2    let text = $state("Initial content");
3
4    $effect(() => {
5      localStorage.setItem("autosave", text);
6    });
7  </script>
8
9  <textarea bind:value={text}></textarea>
```

Runs once on mount, and re-runs whenever 'text' changes

When you use the `$effect` rune, you do not need to explicitly declare your dependencies. Svelte's compiler automatically tracks that `text` was read inside the function, and wires up the reactivity under the hood.

KEY TAKEAWAY

Never use `$effect` to synchronize internal state (e.g., updating variable `B` when variable `A` changes). That is what `$derived` is for. Only use `$effect` when you are explicitly stepping outside the Svelte reactivity system, like writing to the DOM, calling an API, or using `localStorage`.

Cleanup Functions

Many side effects create ongoing processes that need to be dismantled, such as setting intervals or attaching event listeners to the `window`.

To clean these up, your `$effect` function simply returns another function. This returned function is the **cleanup phase**. Svelte will automatically execute this cleanup function right before the effect runs again, and right before the component is destroyed.



```
1  <script>
2    let count = $state(0);
3
4    $effect(() => {
5      const interval = setInterval(() => {
6        count++;
7      }, 1000);
8
9      return () => {
10         clearInterval(interval);
11         Cleans up the previous interval to prevent memory leaks
12       };
13     });
14  </script>
```

Without the cleanup function, the component would spawn a brand new permanent interval timer every time the effect re-ran, immediately crashing the app.

Summary

When you need to interact with external systems outside of Svelte's reactivity model, you use the `$effect` rune. It runs its code block automatically on mount and whenever its internal reactive dependencies change. To prevent memory leaks when managing ongoing processes like timers or event listeners, the effect should return a cleanup function, which Svelte automatically executes before the next run and upon component destruction.

	<code>\$derived</code>	<code>\$effect</code>
Purpose	Calculating data for the UI	Interacting with the external world
Timing	Runs before the DOM updates	Runs after the DOM updates
Cleanup	Not applicable	Returns a cleanup function

Lecture 16: Mounting the Application

INTERVIEW QUESTION

[> CORE](#)

*You've written an `App.svelte` file.
How does it actually get onto the page?*

Svelte compiles your component to JavaScript, but how does that JavaScript end up rendering on the screen? What actually puts your component into the DOM?

Writing a Svelte component is only half the battle. Once your `.svelte` file is compiled into highly optimized JavaScript, you still need a way to take that code and inject it into the browser's Document Object Model (DOM) so the user can actually see it.

The Blank Canvas Problem

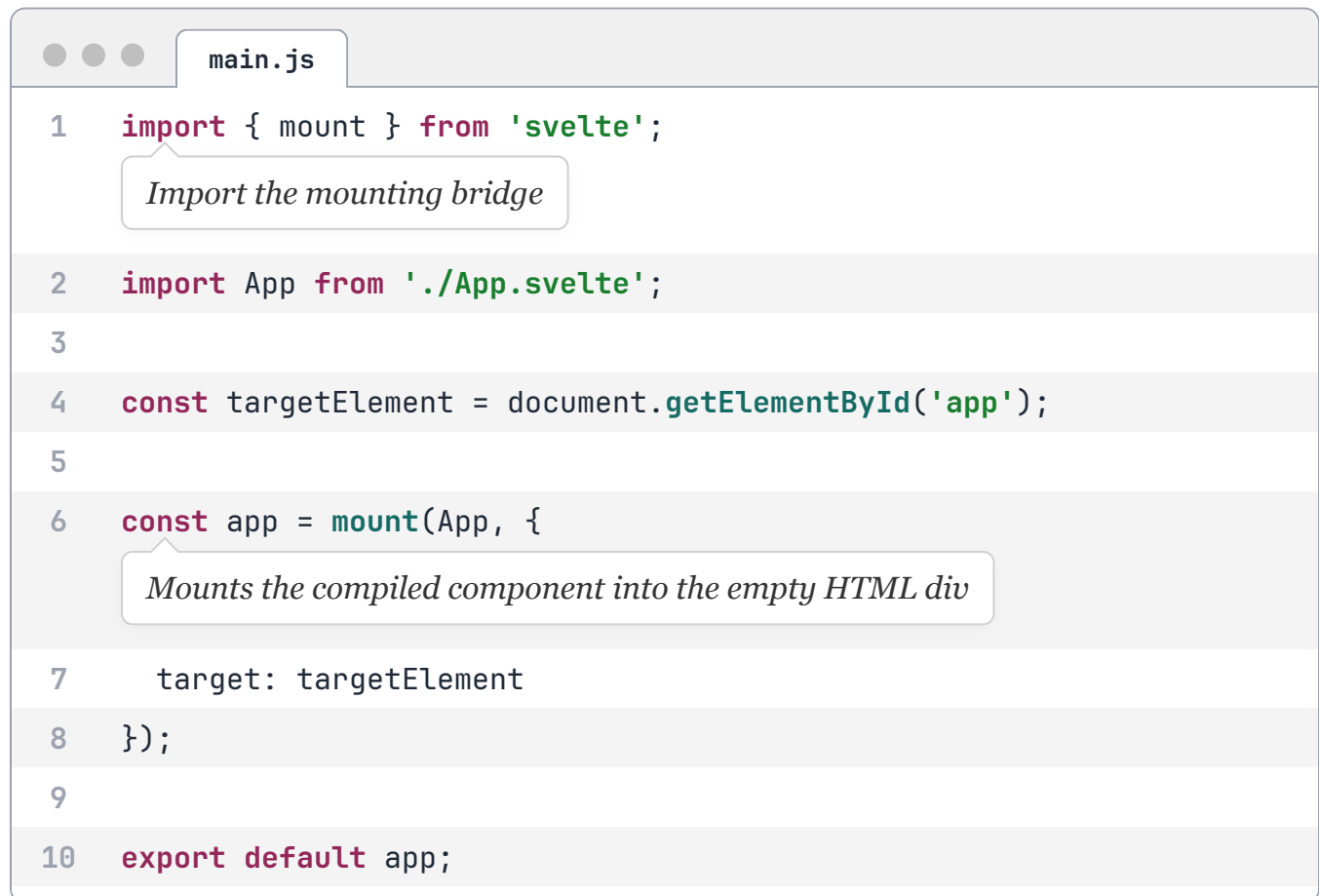
Imagine you have just finished writing the most beautiful `App.svelte` file ever created. You open up your project's `index.html` file in the browser, expecting to see your application, but you are greeted with a completely blank white screen.

If you inspect the HTML file, you'll see a single empty `<div>` with an id of `app`. Your component exists, but the browser has no idea what an `App.svelte` file is. Browsers only understand raw HTML, CSS, and JavaScript. You need a dedicated bridge to take the compiled JavaScript output of your Svelte component and physically append it to that empty `<div>`. If you skip this step, your app remains trapped as raw code in memory forever.

The `mount` Function

In Svelte 5, the bridge between your compiled components and the browser's DOM is the `mount` function.

This function takes two arguments: the root component you want to render (usually `App.svelte`), and the physical DOM element on the page where it should be injected (the target). This code typically lives in a single entry-point file, usually named `main.js` or `main.ts`.



```
1  import { mount } from 'svelte';
2  import App from './App.svelte';
3
4  const targetElement = document.getElementById('app');
5
6  const app = mount(App, {
7    target: targetElement
8  });
9
10 export default app;
```

Import the mounting bridge

Mounts the compiled component into the empty HTML div

The Initial Handshake

When you call `mount`, Svelte executes the `<script>` block of your `App.svelte` component for the very first time. It evaluates the initial values of your `$state` variables, computes any `$derived` values, and constructs the initial DOM nodes. It then grabs that newly constructed HTML and physically appends it as a child into your `target` element.

From that moment onward, the application is alive. The Svelte reactivity system takes over, and any further changes to the state will surgically update the DOM without ever needing to call `mount` again.

INTERVIEW TIP

To impress the interviewer: Note that in Svelte 5, `mount` completely replaces the old Svelte 4 class instantiation method (`new App({ target })`). The new functional `mount` API aligns perfectly with modern JavaScript patterns and makes the framework code significantly easier to tree-shake and optimize.

Summary

A `.svelte` file cannot render itself; it must be explicitly injected into an existing HTML document. To bring your application to life, you use the `mount` function exported by the `svelte` package. By providing your root component and a target DOM element (like an empty `<div>`), `mount` executes the component's setup logic, constructs the initial HTML, and permanently attaches your reactive application to the browser window.

	SVELTE 4 (Legacy)	SVELTE 5 (Modern)
API Style	Class instantiation (<code>new App()</code>)	Functional invocation (<code>mount()</code>)
Target Setup	Passed in options object	Passed in options object
Tree-shaking	Harder for bundlers to optimize	Excellent optimization

Lecture 17: Scoped CSS

INTERVIEW QUESTION

[> CORE](#)

How does Svelte keep one component's CSS from affecting other components?

You add `.button { color: red }` to one component, and you're terrified it'll turn every button in the app red. Does it? Why or why not?

CSS is notoriously difficult to manage at scale. Because traditional CSS is inherently global, a tiny styling change in one file can inadvertently destroy the layout of a completely unrelated page.

The Global Scope Nightmare

Imagine you are hired to fix a bug in a massive e-commerce application. You open the `Header.svelte` component and notice a login button that is supposed to be red. You add a simple `<style>` block at the bottom of the file with the rule `.btn { color: red; }`.

In a traditional global CSS architecture, this is a ticking time bomb. The moment that header loads, the browser applies that `.btn` rule globally across the entire DOM. Suddenly, the "Add to Cart" button in the product grid turns red. The "Delete Account" button in the settings panel turns red. You tried to fix one button, and in the process, you accidentally broke the design system for the entire application, infuriating the design team and causing massive visual confusion for users.

Automatic CSS Scoping

Svelte completely eliminates this nightmare by scoping your CSS to the component by default. When you write a `<style>` block inside a `.svelte` file, those styles only apply to the HTML elements directly inside that exact same file.



```
1  <button class="btn">Login</button>
2
3  <style>
4    .btn { /* ✓ This rule only affects the button in this specific file */
5      color: red;
6    }
7  </style>
```

When you render this component, Svelte guarantees that no other `.btn` on the page will turn red. You can safely reuse class names like `.container`, `.card`, or `.btn` in dozens of different components without ever worrying about them colliding.

How the Magic Works

Svelte achieves this encapsulation during the compilation step. It doesn't rely on complex JavaScript runtime libraries like CSS-in-JS. Instead, it alters your code before it ever reaches the browser.

When the Svelte compiler sees your component, it generates a unique hash string (like `svelte-1xyz5`). It then quietly attaches that hash as an extra class to your HTML element, and rewrites your CSS rule to specifically target that exact hash.

Your source code: `<button class="btn">` + `.btn { color: red }`

What the browser actually receives: `<button class="btn svelte-1xyz5">` + `.btn.svelte-1xyz5 { color: red }`

Because Svelte mathematically guarantees that no other component will ever receive the `svelte-1xyz5` hash, it is physically impossible for the styles to leak out and infect other elements on the page.

INTERVIEW TIP

To impress the interviewer: If you ever explicitly want a style to break out of the component and apply globally (like setting a universal font family on the `<body>`), you can use the `:global()` modifier. E.g., `:global(body) { margin: 0; } .`

Summary

To solve the fragility of global stylesheets, Svelte scopes CSS to the component level by default. The CSS you write inside a `.svelte` file will only ever affect the HTML elements inside that exact file. Behind the scenes, the Svelte compiler enforces this by generating a unique hash class and injecting it into both your HTML tags and your CSS selectors, ensuring your styles are permanently isolated and completely immune to global collisions.

	GLOBAL CSS (Legacy)	SCOPED CSS (Svelte)
Leakage	High risk of unintended changes	Zero risk of styles leaking
Class Naming	Requires complex BEM conventions	Use simple names (<code>.btn</code>) safely
Implementation	Native browser behavior	Compiler-generated hash classes

Lecture 18: Toggling Classes

INTERVIEW QUESTION

[> CORE](#)

How do you apply and toggle classes conditionally in Svelte?

You want a `disabled` class when the button is disabled, an `active` class when it's pressed. How do you toggle classes based on state?

User interfaces are highly dynamic. When a user interacts with a component, it often needs to visually respond by changing its appearance, which usually requires adding or removing CSS classes dynamically.

The Imperative Class Problem

Imagine you are building a custom `<ToggleSwitch>` component for a dark mode setting. When the state variable `isDark` becomes true, you want the switch to turn blue by applying the CSS class `active`.

In older vanilla JavaScript patterns, you would write imperative code to hunt down the DOM element and manually manipulate its class list. You would write

```
document.getElementById('switch').classList.add('active')
```

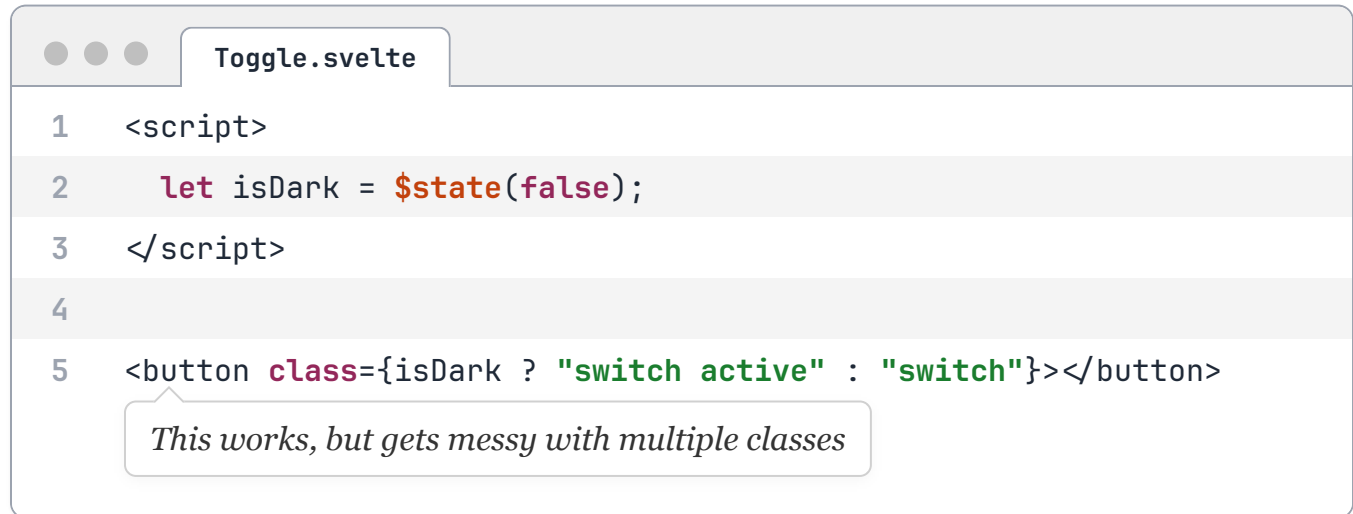
when the setting turned on, and

```
classList.remove('active')
```

when it turned off. This constant, manual synchronization between your JavaScript state and your DOM classes is tedious, prone to typos, and extremely easy to accidentally leave out of sync, resulting in a UI that lies to the user about its true state.

Dynamic Class Binding

Because Svelte's curly braces can evaluate any JavaScript expression, you can theoretically construct a class string dynamically using a ternary operator.



The screenshot shows a code editor window titled "Toggle.svelte". The code is as follows:

```
1 <script>
2   let isDark = $state(false);
3 </script>
4
5 <button class={isDark ? "switch active" : "switch"}></button>
```

A callout box points to the `class` attribute in line 5, containing the text: *This works, but gets messy with multiple classes*

While this works perfectly for a single condition, it quickly devolves into an unreadable mess if you need to toggle three or four different classes simultaneously.

The `class:` Directive

To solve this, Svelte provides a specialized, elegant shorthand called the `class:` directive. It allows you to toggle a specific CSS class on or off based entirely on a boolean condition.

The syntax is `class:name={condition}`. If the condition evaluates to true, Svelte adds the class. If it evaluates to false, Svelte removes it.



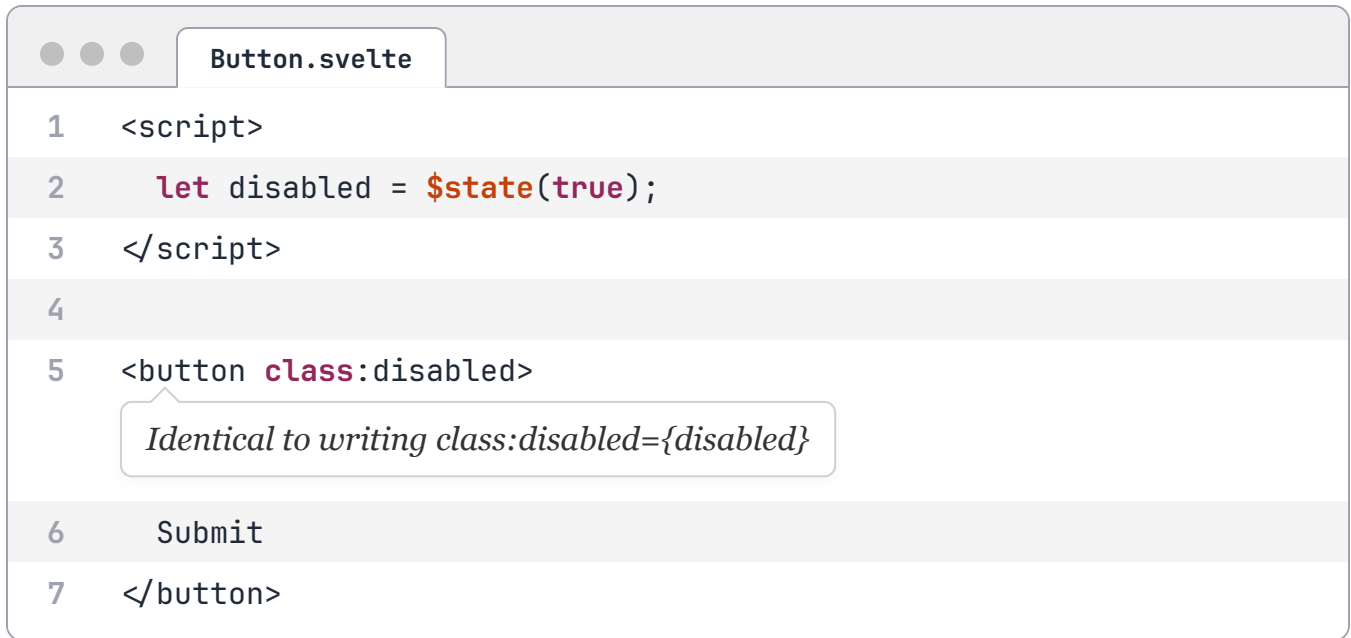
```
1 <script>
2   let isDark = $state(false);
3 </script>
4
5 <button
6   class="switch"
7   class:active={isDark}
8 >
9   Toggle
10 </button>
```

Adds 'active' only when isDark is true

This directive does not overwrite your static `class="switch"`. Svelte intelligently merges them together. If `isDark` is true, the browser receives `class="switch active"`.

The Name-Matching Shorthand

Just like the shorthand for regular attributes, Svelte provides a shortcut for the `class:` directive. If the name of the CSS class you want to toggle is exactly identical to the name of the variable controlling it, you can drop the `=`.



```
1 <script>
2   let disabled = $state(true);
3 </script>
4
5 <button class:disabled>
6   Submit
7 </button>
```

Identical to writing `class:disabled={disabled}`

KEY TAKEAWAY

The `class:` directive is evaluated reactively. You never have to manually add or remove the class using JavaScript; the Svelte compiler handles the DOM updates automatically whenever the underlying `$state` changes.

Summary

To visually adapt your UI based on reactive state, Svelte provides the `class:` directive. Instead of manually pushing strings into a `classList` array, you declare a declarative binding using `class:name={condition}`. Svelte automatically adds or removes the specified class whenever the condition changes, seamlessly merging it with any static classes you have defined. When the class name and the variable name match perfectly, you can use the ultra-clean `class:name` shorthand.

	VANILLA JS (Imperative)	SVELTE (Declarative)
Adding	<code>el.classList.add('active')</code>	<code>class:active={true}</code>
Removing	<code>el.classList.remove('active')</code>	<code>class:active={false}</code>
Shorthand	N/A	<code>class:active</code>

Lecture 19: Dynamic Inline Styles

INTERVIEW QUESTION

[> CORE](#)

How do you set inline styles dynamically, including CSS custom properties?

You need an element's color to come from a variable, or a CSS variable like `--columns` to be set from JavaScript. How do you drive inline styles from state?

While Svelte's `class:` directive is the best tool for toggling predefined styles, there are times when styles must be highly granular or completely unpredictable, such as setting a height to a user's exact mouse coordinate, or applying a custom brand color pulled from an API.

The Problem with Style Strings

Imagine you are building a dashboard where the user can pick a custom hex color for their profile banner.

If you try to achieve this using raw JavaScript DOM manipulation or raw HTML string concatenation, it quickly becomes error prone. In older frameworks, you might have to assemble strings manually like `style="background-color: " + userColor + "; opacity: " + currentOpacity`. This is visually messy, easy to break with a missing semicolon, and highly susceptible to malicious code injection if the user inputs something unexpected instead of a color.

The `style:` Directive

Svelte provides a `style:` directive that allows you to safely and declaratively bind reactive JavaScript variables directly to specific CSS properties.

The syntax is `style:property={value}` .



```
1 <script>
2   let themeColor = $state("#ff0000");
3   let opacity = $state(0.8);
4 </script>
5
6 <div
7   class="banner"
8   style:background-color={themeColor}
9   style:opacity={opacity}
10  >
11   User Profile
12 </div>
```

Binds background to the hex code

When `themeColor` or `opacity` change, Svelte surgically updates only those specific CSS properties on the element's inline style object, leaving any other styles completely untouched.

CSS Custom Properties (Variables)

The true superpower of the `style:` directive is how seamlessly it integrates with CSS Custom Properties (often called CSS variables).

Imagine you are building a dynamic grid layout where the user can choose how many columns they want via a slider. Instead of trying to write inline styles for every single grid child, you can use the `style:` directive on the parent container to define a CSS variable.

```
Grid.svelte
1  <script>
2    let columns = $state(3);
3  </script>
4
5  <div class="grid" style:--col-count={columns}>
    Sets the CSS variable natively on the parent container
6    <div class="item">1</div>
7    <div class="item">2</div>
8    <div class="item">3</div>
9  </div>
10
11  <style>
12    .grid {
13      display: grid;
14      grid-template-columns: repeat(var(--col-count), 1fr); /* ✓ The CSS re
15    }
16  </style>
```

This pattern is incredibly powerful. It allows your JavaScript to safely pass data over the fence to your CSS, letting the CSS engine do what it does best (layout and rendering) without polluting your markup with massive inline style strings.

Summary

When styles cannot be handled by static CSS classes because they rely on exact numerical values or dynamic user data, use Svelte's `style:` directive. It allows you to bind reactive state directly to CSS properties, or, more powerfully, to CSS Custom Properties (variables). This bridges the gap between your JavaScript state and your stylesheets safely and declaratively.

	RAW STRINGS (Legacy)	STYLE DIRECTIVE (Modern)
Syntax	<code>style="color: {val}"</code>	<code>style:color={val}</code>
Readability	Messy and error prone	Clean and declarative
CSS Variables	Difficult to concatenate	Natively supported via <code>style:--var</code>

Lecture 20: Two-Way Component Binding

INTERVIEW QUESTION

> CORE

How do you let a parent read and change a value that lives inside a child component?

Your `Input` component owns its value, but the parent needs to read and reset it. How do you set up two-way binding across the component boundary?

You already know how to pass data down to a child using normal props, and you know how to bind data to native HTML elements like `<input>` using `bind:value`. But what happens when you create your own custom components?

The Stagnant Parent Problem

Imagine you are building a custom `<PinCode>` component for a security application. Inside the component, you have the logic to handle four separate number inputs and concatenate them into a single `pin` string.

You import this `<PinCode>` into your `Login.svelte` parent component. The parent has a "Submit" button, but it cannot see the `pin` string because it is trapped inside the child's local state. Even worse, if the user enters the wrong PIN, the parent needs a way to wipe the child's input clean. If data strictly flows downwards, the parent is completely blind to the child's internal state and powerless to reset it.

The `$bindable()` Rune

In Svelte 5, you can explicitly mark a child component's prop as open for two-way communication using the `$bindable` rune.

By default, props are read-only. If the parent passes a value down, the child cannot alter it. But when the child wraps a prop in `$bindable()`, it declares: *"I am allowed to change this value, and if I do, the parent's variable will instantly update to match."*



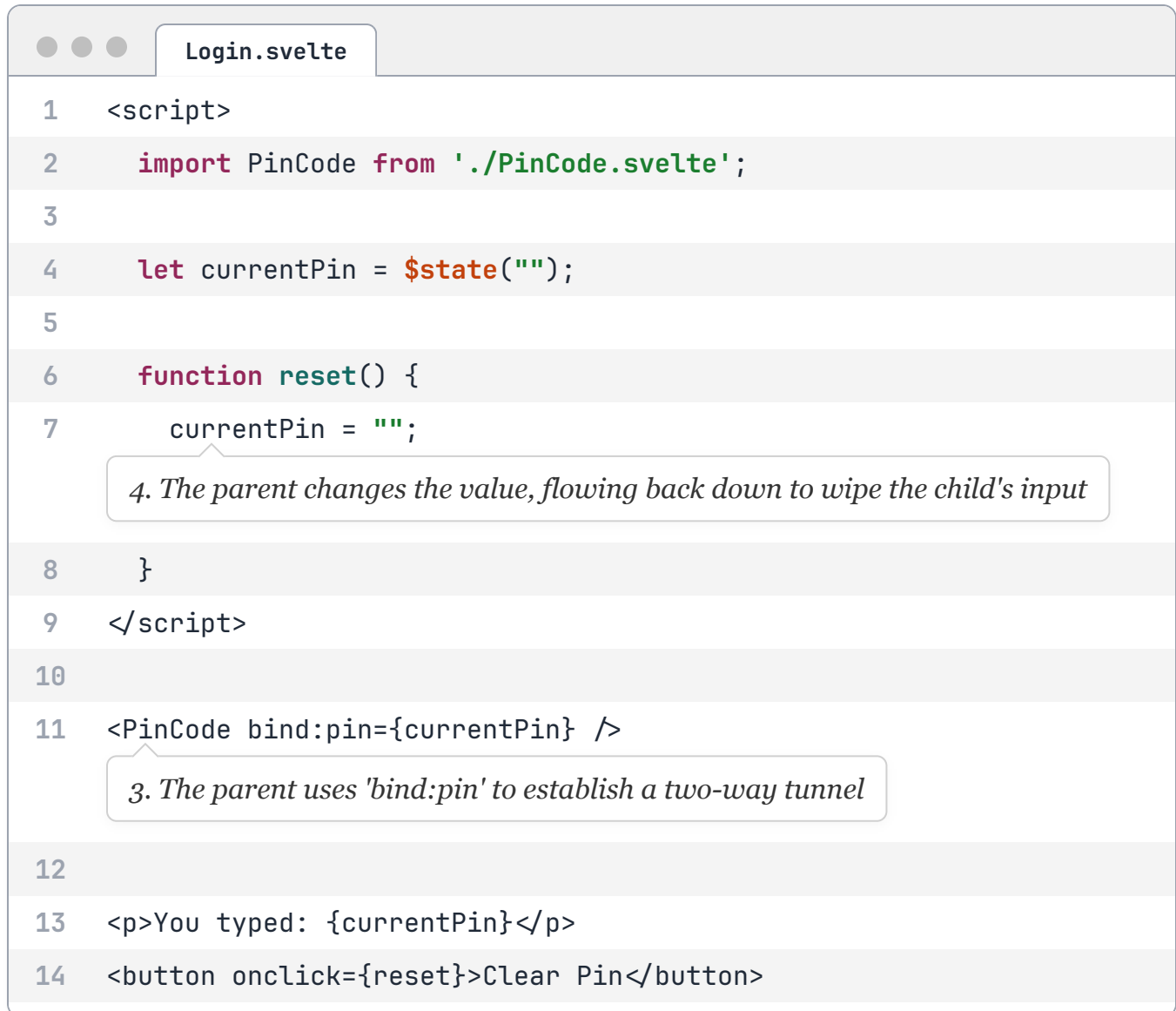
```
1  <script>
2    let { pin = $bindable() } = $props();
3  </script>
4
5  <input type="text" bind:value={pin} />
```

1. The child explicitly marks 'pin' as open for two-way binding

2. The child binds its local HTML input to the bindable prop

Binding Across the Boundary

Now that the child has given permission, the parent component can use the `bind:` directive directly on the custom component tag, exactly as it would on a native HTML `<input>` .



```
1  <script>
2    import PinCode from './PinCode.svelte';
3
4    let currentPin = $state("");
5
6    function reset() {
7      currentPin = "";
8    }
9  </script>
10
11  <PinCode bind:pin={currentPin} />
12
13  <p>You typed: {currentPin}</p>
14  <button onclick={reset}>Clear Pin</button>
```

4. The parent changes the value, flowing back down to wipe the child's input

3. The parent uses 'bind:pin' to establish a two-way tunnel

When the user types inside the child component, the native `<input>` updates the `pin` prop. Because `pin` is `$bindable()` , the change travels up the tunnel and updates the parent's `currentPin` state. When the parent clicks the reset button, `currentPin` changes to an empty string, traveling back down the tunnel to clear the child's `<input>` .

KEY TAKEAWAY

You can provide a default fallback value to a bindable prop just like any other prop: `let { pin = $bindable("0000") } = $props();`

Summary

To allow data to flow seamlessly in both directions across component boundaries, use the `$bindable()` rune. The child component must explicitly opt-in by wrapping its prop declaration in `$bindable()` . Once authorized, the parent component can use the `bind:propName` directive on the custom component. This creates a two-way synchronization tunnel, allowing both the parent and the child to read and modify the shared data safely.

	NORMAL PROPS	BINDABLE PROPS
Declaration	<pre>let { name } = \$props();</pre>	<pre>let { name = \$bindable() } = \$props();</pre>
Data Flow	Parent to Child (One-Way)	Two-Way Synchronization
Parent Usage	<pre><Child name={val} ></pre>	<pre><Child bind:name={val} ></pre>

Lecture 21: Reusable Markup with Snippets

INTERVIEW QUESTION

[> CORE](#)

How do you write a reusable chunk of markup you can use in several places?

The same three lines of markup appear in five spots in your component. Copy-paste feels wrong. Can you name a block of markup and reuse it?

The Five-Copy Disaster

You are building a product page. The same `<figure>` block, an image with a caption and a price badge, has to appear in five spots: the hero gallery, the "Customers also bought" strip, the search preview, the "Recently viewed" shelf, and a share dialog. You copy the five lines into all five places and ship. Three weeks later the designer asks for a loading spinner on every image. You edit the copies, but one was hiding inside an `{#if}` branch you forgot existed, so the "Recently viewed" shelf still renders the old markup. There is no error and no red squiggle. The page just looks broken in one corner, and the only way to catch it is for a human to notice.

That is the failure mode of duplicated markup: the copies drift apart silently, and the cost grows with every copy. The fix is not more discipline. The fix is making sure there is only *one* copy that has a name.

Naming the Block

Notice what the five copies actually are. They are not five features. They are one block of HTML stamped into the page five times, each time with different data. So the job is this: take a block of markup, give it a name, and call that name wherever the block should appear, passing in the data for that spot.

This is exactly what a function does for JavaScript: name a block of statements once, call it by name with arguments. Svelte gives you the same move for markup. That named, callable block of template is called a **snippet**. You define it with `{#snippet ...}` and stamp it into the page with `{@render ...}`.

KEY TAKEAWAY

A snippet is to markup what a function is to logic: name it once, call it anywhere.

Just as a function lets you write a block of JavaScript once and invoke it with different arguments, a snippet lets you write a block of HTML once and render it with different data. The markup lives in exactly one place, so a change to the snippet updates every spot that renders it, with zero copies to hunt down.

ProductPage.svelte

```
1  {#snippet figure()}
```

1. **NAME** *the block of markup once*

```
2    <figure>
```

```
3      
```

```
4      <figcaption>Flagship smartphone</figcaption>
```

```
5    </figure>
```

```
6  {/snippet}
```

```
7
```

```
8  {@render figure()}
```

2. **CALL** *it by name, as many times as you need*

```
9  {@render figure()}
```

Passing Parameters

A snippet that renders one fixed image is barely better than a copy. Its real power appears when you give it **parameters**, named inputs exactly like function arguments, so each call site passes its own data.

● ● ● ProductPage.svelte

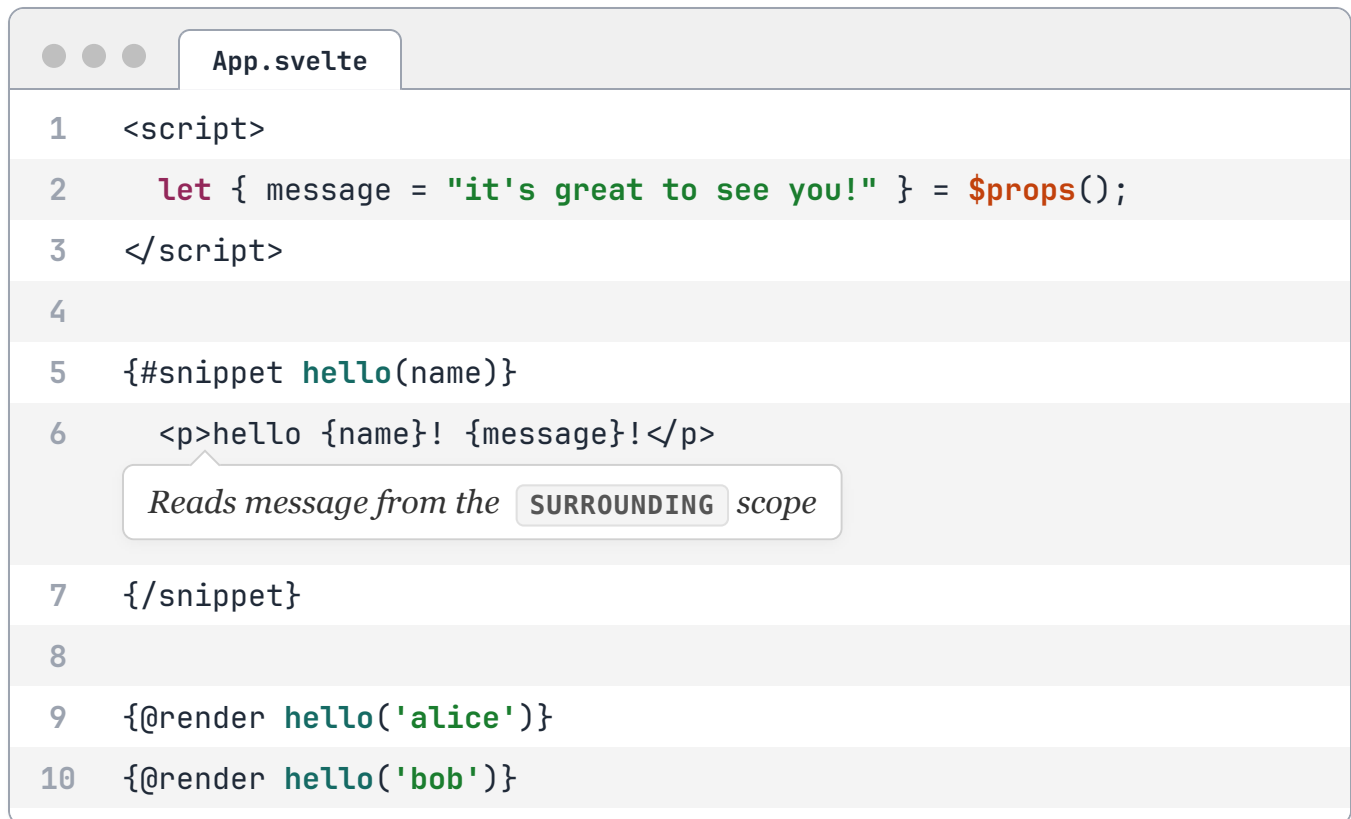
```
1  {#snippet figure(image)}  
    The PARAMETER lets each call pass its own data  
2      <figure>  
3          <img src={image.src} alt={image.caption} />  
4          <figcaption>{image.caption}</figcaption>  
5      </figure>  
6  {/snippet}  
7  
8  {#each products as product}  
9      {#if product.href}  
10         <a href={product.href}>{@render figure(product)}</a>  
    Linked products wrap the figure  
11         {:#else}  
12             {@render figure(product)}  
    Without a snippet this <figure> would be DUPLICATED here  
13         {/if}  
14     {/each}
```

Every product renders the same `<figure>`, but linked products wrap it in an `<a>` while unlinked ones render it bare. Without a snippet you would duplicate the entire `<figure>` in both branches, giving you two copies guaranteed to drift. Like a

function, a snippet can have any number of parameters with default values and destructuring; the one thing you cannot use is a rest parameter.

Snippet Scope

Snippets obey ordinary **lexical scope**: they read any variable visible where they are declared, whether from the `<script>` tag or a surrounding `{#each}` block, with no passing required. They are visible to siblings and their children, but not outside the block they were declared in, and they can even call themselves, so recursive markup is possible.

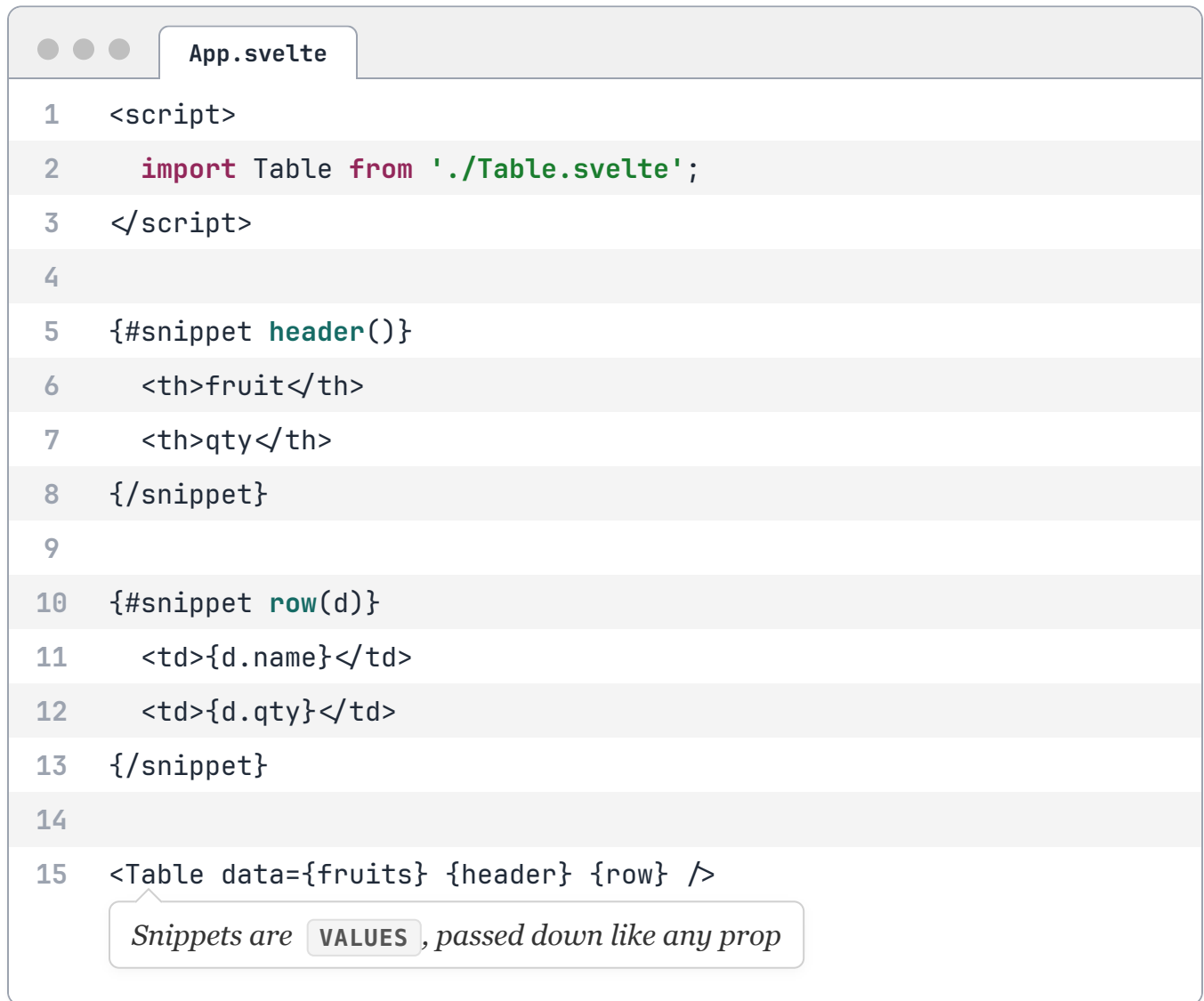


```
1  <script>
2    let { message = "it's great to see you!" } = $props();
3  </script>
4
5  {#snippet hello(name)}
6    <p>hello {name}! {message}!</p>
7  {/snippet}
8
9  {@render hello('alice')}
10 {@render hello('bob')}
```

*Reads message from the **SURROUNDING** scope*

Snippets as Component Props

So far the snippet has stayed inside the component that declared it. The bigger unlock is that a snippet is a *value*, just like a number or a string, so you can hand it to a child as a prop and let the child decide when and where to render it. You are passing *content* down, not *data*.



```
1  <script>
2    import Table from './Table.svelte';
3  </script>
4
5  {#snippet header()}
6    <th>fruit</th>
7    <th>qty</th>
8  {/snippet}
9
10 {#snippet row(d)}
11   <td>{d.name}</td>
12   <td>{d.qty}</td>
13 {/snippet}
14
15 <Table data={fruits} {header} {row} />
```

Snippets are **VALUES**, passed down like any prop

The child receives the snippets through `$props()` and renders them, feeding each row of data into the `row` snippet as it loops (`<tr>{@render row(d)}</tr>`). As a convenience, a snippet declared directly *between a component's tags* becomes a prop on that component automatically, so the attribute syntax is optional.

The `children` Snippet and the End of Slots

Any content between a component's tags that is *not* a named snippet automatically becomes a special snippet called `children`, which is what powers the most common pattern in Svelte: a wrapper that renders whatever you put inside it.

```
1 <script>
2   let { children } = $props();
3 </script>
4
5 <button>{@render children()}</button>
```

Leftover content becomes the **CHILDREN** snippet

Renders whatever the parent put between the tags

Because a snippet prop can be missing, render optional ones defensively with `{@render children?.()}`, or wrap them in an `{#if children}` block with fallback content in the `:else`.

KEY TAKEAWAY

Snippets are the replacement for slots: same job, but a snippet can take parameters and be passed around as a value.

In Svelte 4 you passed content into a component with **slots**, named placeholders a child filled with markup from its parent. Slots worked, but they could not take parameters and were not values you could store or pass. Snippets do everything slots did and more, so slots are deprecated in Svelte 5. One caution

from the slots era: because non-snippet content becomes the `children` snippet, a component cannot also declare a normal prop named `children` , so avoid that prop name.

INTERVIEW TIP

To impress the interviewer: Many candidates can recite that "slots are replaced by snippets." The answer that signals real depth is explaining *what snippets add*: because a snippet is a first-class value that takes parameters, one component can receive several differently-shaped blocks of markup (`header` , `row` , `footer`) and call each with its own data, something slots could never express. That is the mechanical reason the framework made the switch.

Summary

A **snippet** is a named, reusable block of markup inside a component, declared with `{#snippet name()}` and stamped into the page with `{@render name()}`. Snippets take parameters like functions, obey ordinary lexical scope, and are first-class values you can pass into child components as props, where leftover content becomes the special `children` snippet. By collapsing every duplicated copy of a block of markup into one named definition, snippets eliminate the silent drift that copy-paste causes and fully replace the deprecated slots mechanism from Svelte 4.

	COPY-PASTE / SLOTS (The Old Ways)	SNIPPETS (Svelte 5)
Single source	One copy per usage site, drifting silently	One named definition, rendered by name
Parameters	None (slots cannot take arguments)	Any number, with defaults and destructuring

	COPY-PASTE / SLOTS (The Old Ways)	SNIPPETS (Svelte 5)
Pass to child	Slots only, fixed by the child's template	As a prop value: <code>{header}</code> or between the tags
Reuse across files	Not possible	Export from <code><script module></code> (Svelte 5.5+)

Lecture 22: Typing a Component with TypeScript

INTERVIEW QUESTION

[› CORE](#)

How do you use TypeScript inside a `.svelte` component and type its props?

You want type safety in a component. Typed props, typed state. What do you add to the script tag, and how do you declare a prop interface?

The Bug Nobody Could Find

You are building a `<PriceBlock>` component for a shop. It takes a `price` prop, and the parent grid is supposed to hand it a number. One afternoon a teammate wires it up from a different page and, without noticing, passes the price as a string: `price="29.99"` instead of `price={29.99}`.

Nothing errors. The app builds. The page renders. But the little "add insurance" upsell that runs `price + 1` now performs string concatenation instead of addition, so the cart quietly shows `29.991` dollars instead of `30.99`, and a customer screenshots the broken store before anyone on the team notices. The editor never warned you, because plain JavaScript has no idea that `price` was *supposed* to be a number; the mistake sailed straight through the build and detonated in front of a shopper.

That is the failure mode of untyped props: the contract between a parent and a child lives only in the author's head, and a typo or a wrong type becomes a silent runtime bug. The fix is to write that contract down in a form the editor can enforce, so the mistake is caught while you are typing, not after you ship.

The One Line That Turns It On

To bring TypeScript into a Svelte component, you add a single attribute to the script tag: `lang="ts"` .

```
1 <script lang="ts">
2   let name: string = 'world';
3
4   function greet(name: string) {
5     alert(`Hello, ${name}!`);
6   }
7 </script>
```

With that attribute in place, you can use TypeScript's type annotations, its `interface` declarations, and the rest of its type vocabulary throughout the component, and the Svelte tooling plus your editor will check them for you. `svelte-check` does the same job on the command line, so you can run it in CI.

KEY TAKEAWAY

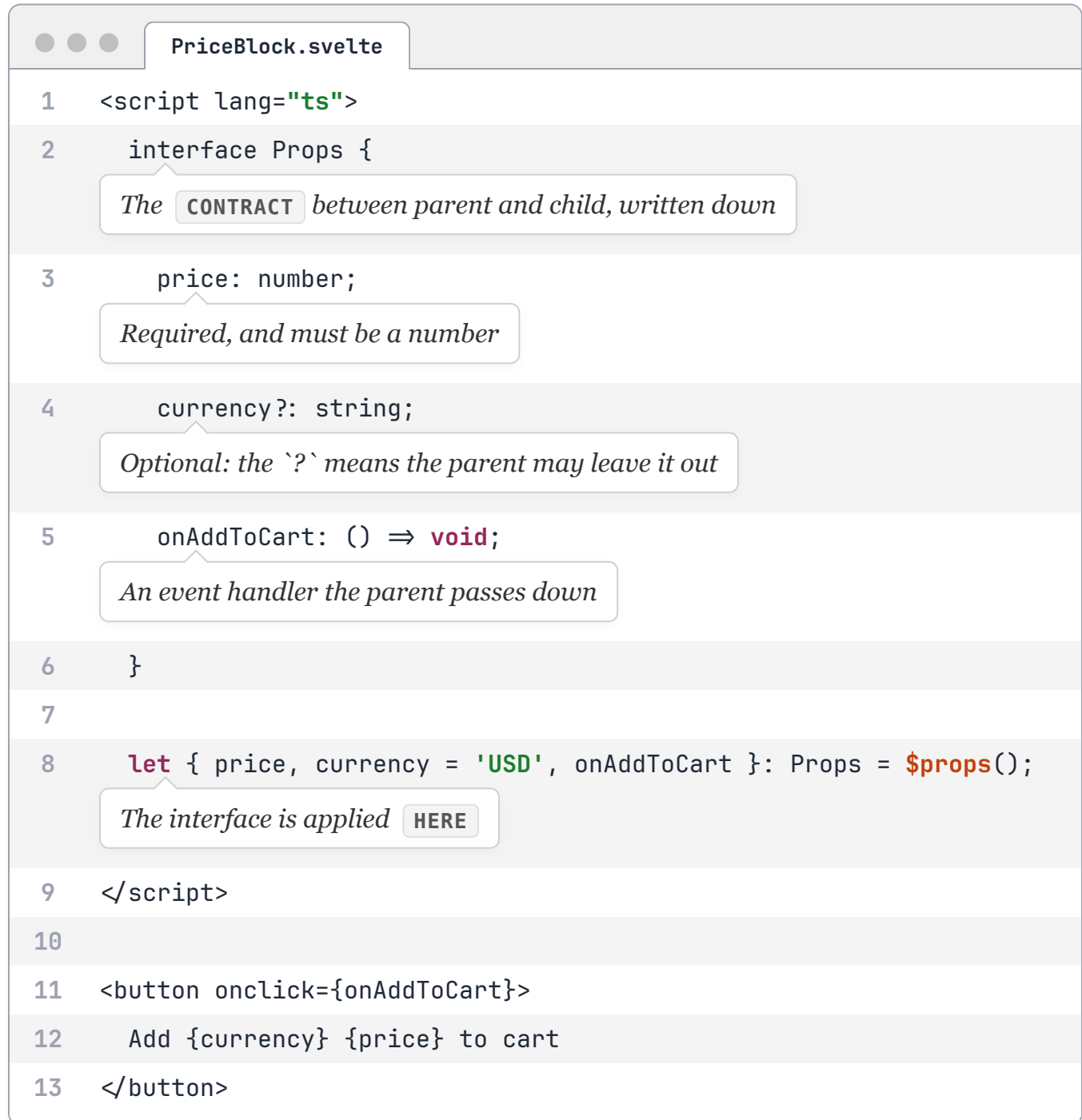
Types are scaffolding, not structure: they guide the build, then vanish without leaving a single byte in the bundle.

Here is the mechanical reason `lang="ts"` needs no extra tooling for the common case. TypeScript has two kinds of features. Type-only features, like annotations and interfaces, describe your code but compile down to nothing: the Svelte

compiler simply strips them out and emits plain JavaScript, so they cost zero bytes and zero runtime. Features that require the TypeScript compiler to *output* real code, like enums, cannot be stripped away, so those need a preprocessor such as `vitePreprocess` to translate them first.

Typing the Props

The direct answer to the interview question is an `interface` describing the component's props, attached to the `$props()` destructuring with a type annotation.



```
1  <script lang="ts">
2    interface Props {
3      price: number;
4      currency?: string;
5      onAddToCart: () => void;
6    }
7
8    let { price, currency = 'USD', onAddToCart }: Props = $props();
9  </script>
10
11  <button onclick={onAddToCart}>
12    Add {currency} {price} to cart
13  </button>
```

PriceBlock.svelte

*The **CONTRACT** between parent and child, written down*

Required, and must be a number

Optional: the `?` means the parent may leave it out

An event handler the parent passes down

*The interface is applied **HERE***

Three things are happening at once. The `interface Props` block declares, prop by prop, what the parent is allowed to

pass: `price` must be a number, `currency` is an optional string with a default, and `onAddToCart` is a function. The `: Props` annotation on the destructuring is what actually binds that contract to the incoming props. And because the editor now knows the shape, the moment a parent writes `price="29.99"` or misspells `prcie`, you get a red squiggle before the file is even saved. The bug from the opening is now impossible to ship.

INTERVIEW TIP

To impress the interviewer: Don't stop at "you add `lang="ts"` and an interface." The answer that signals real depth names *why* it needs no extra tooling: type-only features are erased at compile time, so a typed Svelte component ships the exact same JavaScript as an untyped one. That ties TypeScript directly to Svelte's compiler-first design, and it is the detail that separates a framework user from someone who understands the machinery.

Typing State and Snippets

`$state` is typed like any other variable, though it usually needs no annotation because the initial value already tells TypeScript the type. You spell the type out only when it is wider than the initial value, as in `let note: string | null = $state(null)`, and if you give `$state` no initial value at all, an `as` cast states what it will be once set, which is common inside classes. Snippet props are typed with the `Snippet` interface from `svelte`, where the type argument is a tuple of the snippet's parameters: `row: Snippet<[Product]>` (see Lecture 21 for how snippets work).

The Preprocessor Boundary

Type-only features work the moment you add `lang="ts"`.

The moment you reach for a feature that emits JavaScript, an enum being the classic example, the compiler needs help. In a Vite or SvelteKit project you hand that off with one line of config:



```
1  import { vitePreprocess } from '@sveltejs/vite-plugin-svelte';
2
3  const config = {
4    preprocess: vitePreprocess({ script: true })
5  };
6
7  export default config;
```

*Lets TypeScript **EMIT** code, not just erase it*

In practice, most component code lives entirely in type-only territory, so the plain `lang="ts"` attribute is all you ever touch day to day.

A First Step into Generics

Sometimes the props relate to each other. A list component receives `items` and a `select` callback, and the two must agree on the type of a single item. You express that relationship with the `generics` attribute on the script tag, and the editor then enforces that both sides match for whoever uses the component. This is deeper than the core question, but it is the natural next step once props are typed.

Summary

To use TypeScript in a Svelte component, add `lang="ts"` to the script tag, then declare an `interface Props` describing each prop and attach it to the props destructuring with `let { ... }: Props = $props()`. This works with no extra setup for type-only features, because the Svelte compiler strips types away at build time, leaving the bundle byte-for-byte identical to plain JavaScript; only code-emitting features like enums need a preprocessor such as `vitePreprocess`. State is typed like any variable, snippet props use the `Snippet` interface, and the editor plus `svelte-check` enforce the whole contract so prop mistakes surface at authoring time instead of in production.

	PLAIN JAVASCRIPT (Runtime Discovery)	TYPESCRIPT (Authoring-Time Checks)
Prop mistakes	Silent runtime bug, found by users	Red squiggle the moment you type it
The contract	Lives in the author's head	Written as an <code>interface</code> <code>Props</code>

	PLAIN JAVASCRIPT (Runtime Discovery)	TYPESCRIPT (Authoring-Time Checks)
Setup	None	Add <code>lang="ts"</code> to the script tag
Bundle cost	N/A	Zero: types are erased at build time